19/9/20

Linked list

1) Dynamic collections

21/9/20

~ ⇒ 1's complement

How to remeber precedence of operators

1. 1st row remember yourself
2. U (Unary)
3, 4 a (Authmetic)
5 s (shift)
6, 7 C (conditional)
8, 9, 10 b (Bitwise)
11, 12, 13 l (logical)
14 Ternary
15 a (assignment)
16 c (commom)

U are c blac
You ask Black

Scanned with CamScanner

eak 2004

Q. 2 matrices M_1 & M_2 are to be stored in arrays A & B respectively. each array can be stored either in RMO & CMO in contiguous memory locations. The time complexity of an "algorithm" to compute M_1 & M_2 will be

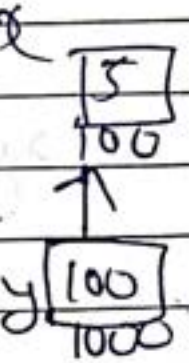
a) Best if A is in RMO & B is in CMO
b) Best if both are in RMO
c) Best if both are in CMO
d) Independent of storage scheme.

Implementation

(a)

In case of implementation, A in RMO, B in CMO

```
int x = 5
int *y = &x;
printf("%d", x);    — 5
printf("%u", &x);   — 100
printf("%u", y);    — 100
printf("%d", *y);   — 5.
printf("%u", &y);   — 1000
```

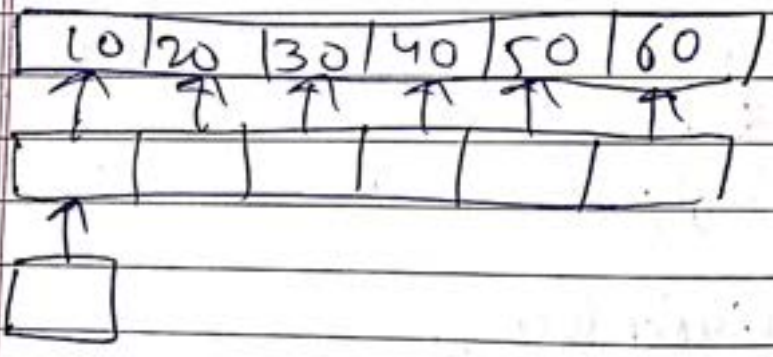


a[3] user friendly
*(a+3) machine friendly
(a+3) = &a[3];

Scanned with CamScanner

```
main(){
1  int a[] = {10, 20, 30, 40, 50, 60};
2  int *p[] = {a, a+1, a+2, a+3, a+4, a+5};
3  int **pp = p;
4  pp++;
5  printf("%d, %d, %d", pp-p, *pp-a, **pp);
6  *pp++;
7  printf("%d, %d, %d", pp-p, *pp-a, **pp);
8  ++*pp;
9  printf("%d, %d, %d", pp-p, *pp-a, **pp);
10 ++**pp;
11 printf("%d, %d, %d", pp-p, *pp-a, **pp);
}
```

P = array of pointers (∵ [] more precedence)



4 pp points to p+1

5 pp-p = 1 (∵ pp points to p+1 and p, i.e., p)

**pp = a[1] ∵ [] has more precedence than *

a = a so *pp - a = 1

**pp = 20

O/P of 5 is // 1 1 20

6. *p p++;

* & ++ have same precedence
Associativity R→L

So *(pp++);

*(p+2) ∴ 3

7) pp-p = 2 now ∴ pp = p+2
p = p

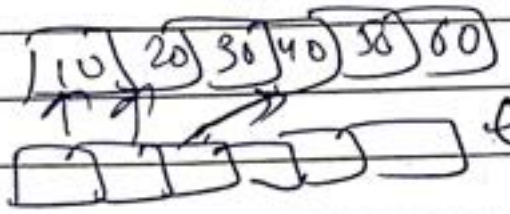
*pp = a[2]
a = a *pp-a = 2

**pp = 30

O/P of 7 is // 2 2 30

8) ++ *pp Precedence ++(*pp);
Associativity

So



9) pp-p = 2 again ∴ Visualize
*pp-a = 3
**pp = 40
O/P // 2 3 40

Scanned with CamScanner

10 ++***pp;

⇒ (++(*(*pp)))

46 41

*pp-p = 2

**pp-a = 3

***pp = 41

O/P // 2 3 41

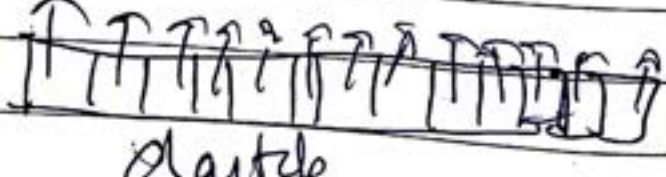
→ char **argv;



→ int (*daytab)[13];



→ in *daytab[13];



Scanned with CamScanner

→ void *comb()

() high precendence, so comb() is a function having return type void *

→ char (*comp)()

comp is a pointer to a function which returns a character.

→ char (*(*f())[])()

f → function f() which returns a pointer *f()

to an array (*f())[]

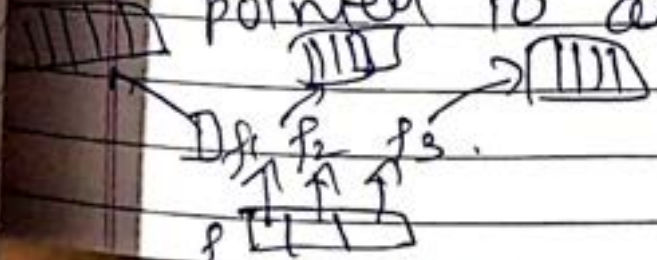
of pointer *(*f())[]

to a function which returns char

→ char (*(*f[3])())[5]



f → array of 3 elements which are pointer to the function returning pointer to array of 5 char



Scanned with CamScanner

```
struct ex{
int i;
char c;
};
```

```
struct ex1{
struct ex a;
struct ex b;
};
```

```
struct ex1 t;
t.a.i = 10;
t.a.c = 'c';
```

22/9/20 → Don't depend, if you put 2x in all machines

malloc() → function call

sbrk() → system call

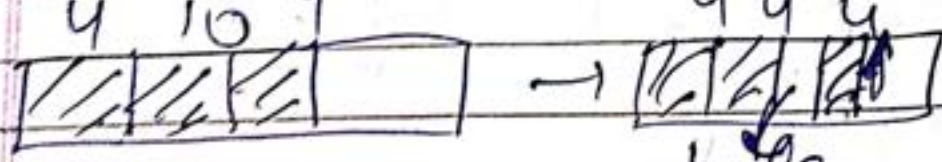
↓

Depends on system

(If you write 2, 4 you are not sure, it depends on system)

malloc calls sbrk(n) inside it

Now



holes

(Memory management

Malloc → first fit (Remember OS)

Scanned with CamScanner

Date
Page 121

IP → Instruction pointer
points to statement to be executed

Use in recursion (stack in general)

for SLL
1) Traversing
2) Insert
3) Delete

```
int *p;
p = 0 valid
int *p = 0 valid
        dNull
```

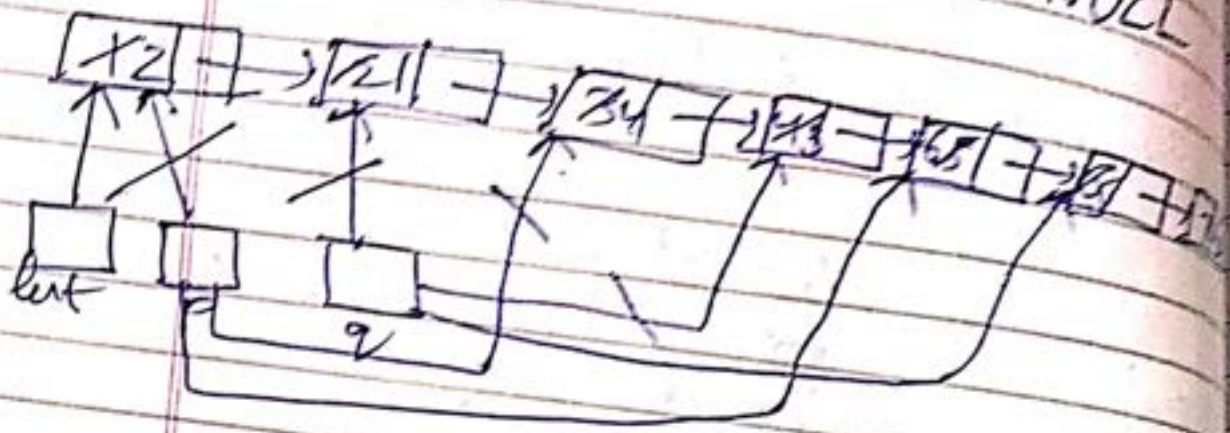
for LL 1, 2, 3, 4, 5, 6, 7

```
struct node {
  int val;
  struct node *next; };
void rearrange (struct node *list) {
struct node *p, *q;
  int temp;
if (!list || !list->next) return;
p = list; q = list->next;
while (q)
{ temp = p->val; p->val = q->val;
q->val = temp; p = q->next;
  q = p ? p->next : 0;
}
```

a) 1, 2, 3, 4, 5, 6, 7 b) 2, 1, 4, 3, 6, 5, 7
c) 1, 3, 2, 5, 4, 7, 6 d) 2, 3, 4, 5, 6, 7, 1

Scanned with CamScanner

!list == list == NULL
!list->next == list->next == NULL



[2, 1, 4, 3, 6, 5, 7 (B)]

Code:

2010 - Put node from last & put it to beginning

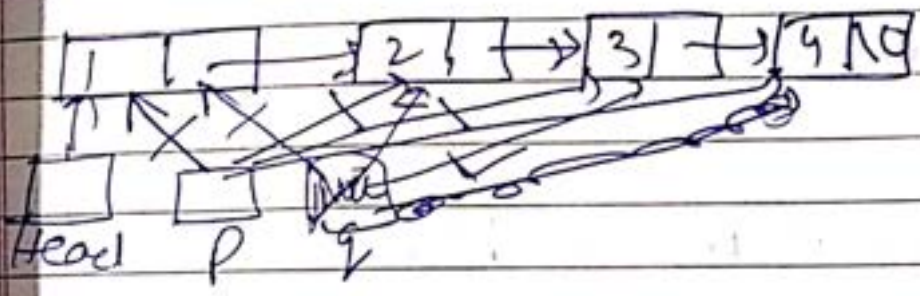
```
typedef struct node {
    int value;
    struct node *next;
} Node;
Node * move_to_front(Node *head)
{
    Node *p, *q;
    if (head == NULL || (head->next == NULL))
        return head;
    q = NULL; p = head;
    while (p->next != NULL) {
        q = p;
        p = p->next;
    }
    return head;
```

Scanned with CamScanner

Choose the correct alternative to replace the blank

a) q = NULL; p->next = head; head = p;
b) q->next = NULL; head = p; p->next = head;
c) head = p; p->next = q; q->next = NULL;
d) q->next = NULL; p->next = head; head = p;

Ans Whenever this kind of Q asked, just assume any 4 node linked list, it will be helpful



c) Ans

Recursion & SLL

1) void f(struct node *p){
if(p){
printf("%d", p->data);
f(p->link);
}
}

Correct order 1234

2) void f(struct node *p){
if(p){
f(p->link);
printf("%d", p->data);
}
}

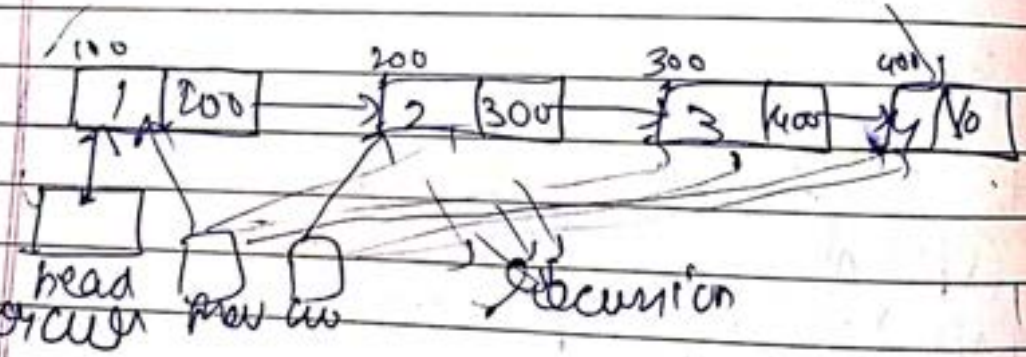
Reverse order 4321

Scanned with CamScanner

Recursive version to reverse a SLL

```
struct node *head;
void reverse(struct node *prev,
        struct node *cur) {
if (cur) {
  reverse(cur, cur->link);
  cur->link = prev;
}
else
  head = prev;
}
void main() {
  reverse(NULL, head);
}
```

Explanation



When cur = NULL, curr->link = prev;
head point to prev

as before when curr->link = prev;
So from back nodes start to point to front &
& at last cur = head = prev;

Scanned with CamScanner

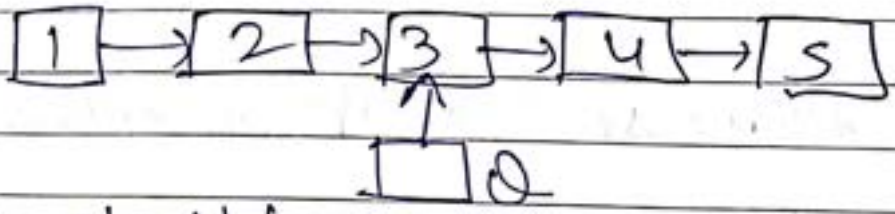
Gate 2004

Q→ In the worst case, the # of comparisons needed to search a singly linked list of length 'n' for a given element is
a) $\log_2 n$ b) n/2 c) $\log_2 n - 1$ d) n

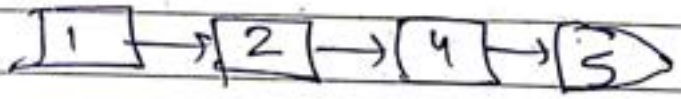
∴ linear search

Q→ Let 'P' be a singly linked list, and 'Q' be a pointer to an intermediate node 'x' in the list. What is the worst-case time complexity of the best-known algorithm to delete the node 'x' from the list?

Ans→



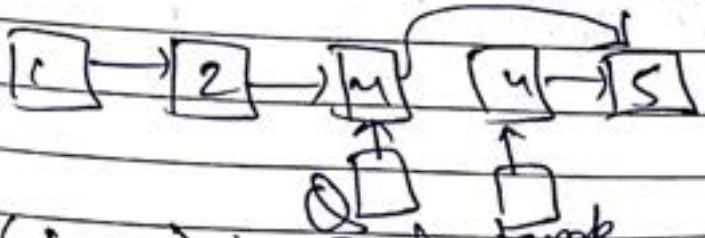
New LL should be



So we do Q→data = Q→link→data
So it becomes



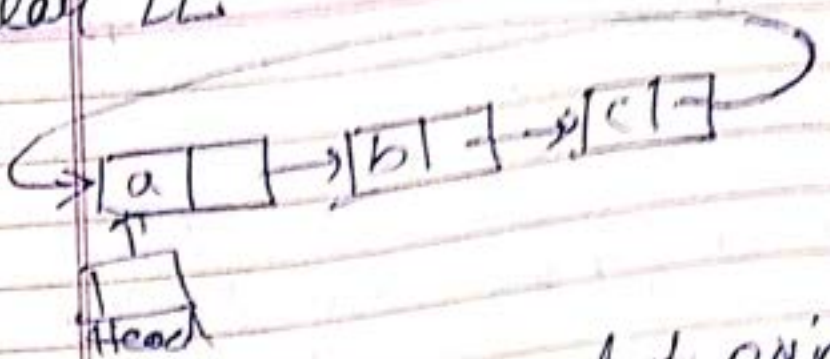
*temp = Q→link;
Now Q→link = Q→link→link
so



free(temp); that's it. So O(1).
temp = NULL;

Scanned with CamScanner

Circular LL.

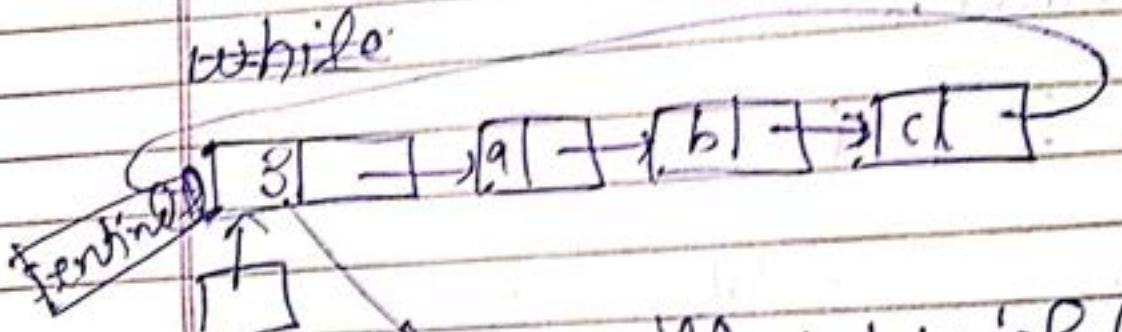


Adv 1) Not wasting last pointer
2) Traverse both. Completely

Disadv 1) We don't know what the end of list if head not specified.

How to traverse (Concept of Sentinel)

while



This will contain info like count of nodes or any info which tells that this is head node etc.
This node is called sentinel

Sentinel may be either at beginning or end.

Scanned with CamScanner

Gate 2003

Date | Page | 127

```
struct item {
    int data;
    struct item *next;
};
int f(struct item *p) {
return ((p==NULL || p->next==NULL ||
((p->data <= p->next->data) && f(p->next));
}
```

Ans a || b || (c && d)

Please remember the concept of short circuiting.

a means no item (element) in LL

b " 1 " " " "

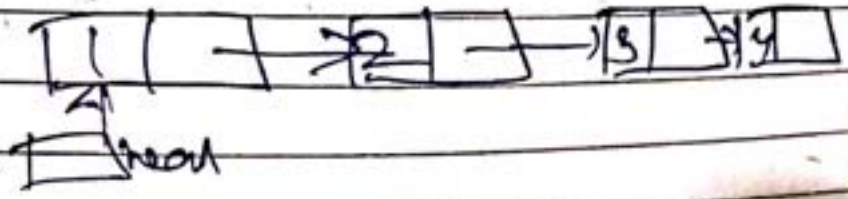
c 2nd elemt data >= 1st elemt data

d p->next & recursion

So function checks if list is in increasing order of data or not.

return True (1) if so
else false (0)

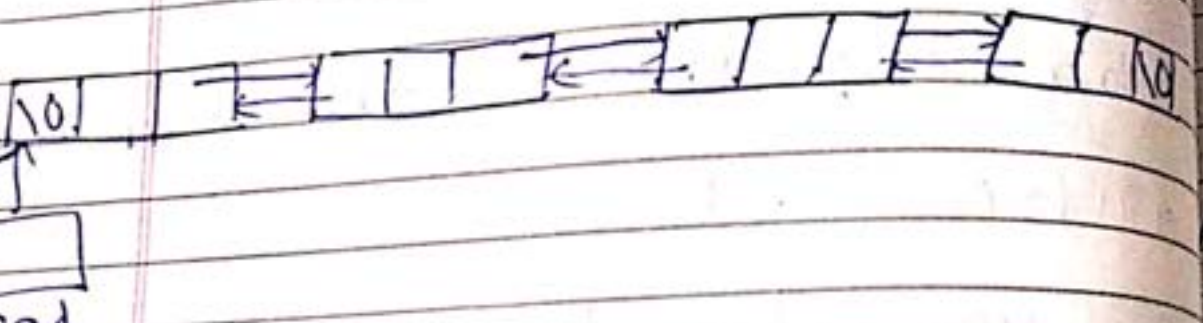
Also analyze by



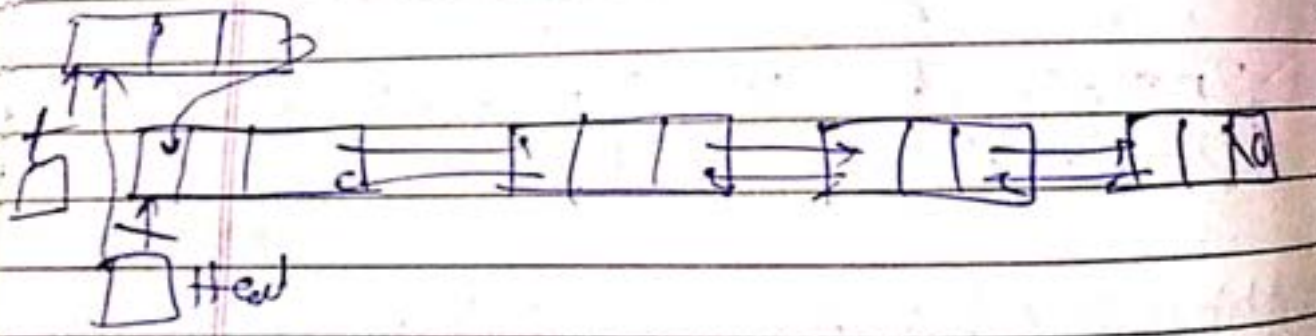
Scanned with CamScanner

Doubly LL

```
struct node{
int i;
struct node *prev;
struct node *next;
};
```



Let's insert node (i) at beginning of head




```
t -> next = head;
head = t
t -> prev = NULL
head -> next -> prev = head;
```

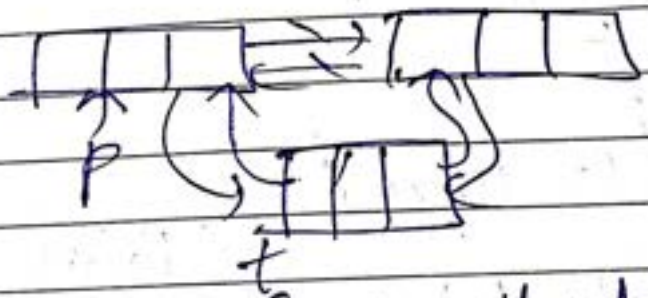
Scanned with CamScanner

(ii) at end

```
p = head;
while(p->next!=
    p = p->next;

p->next = t;
t->prev = p;
t->next = NULL;
```

(iii) Intermediate



Traverse from start to p, then

```
t->prev = p;
t->next = p->next
p->next = t
t->next->prev = t;
```

→ LIFO or FILO

Stacks Used in →

→ Recursion
→ If → PF Conversion (Infix → Postfix)
→ Parsing → matching parentheses, match tags <>
→ Browsing
→ Undo/Redo in MS word (Editors)
→ Tree traversals & graph traversals

Scanned with CamScanner

→ Always remember → whatever DS you learn the implementation can be done only using hold on array.

Implementation

(i) using arrays

```
int stack[INT_MAX];
int top=-1; //Stack is empty
```




```
void push(int item){
if(top==MAX-1)
  printf("overflow");
else{
  top=top+1;
  stack[top]=item;
}
```

wrong we won't follow this

or `stack[top++]=item;` ✗

or `stack[++top]=item;` ✓

```
void pop(){
  int temp;
if(top==-1)
  printf("Underflow");//or return -1;
else{
  top=top-1;
  temp=stack[top];
  top=top-1;
return temp;
}
```

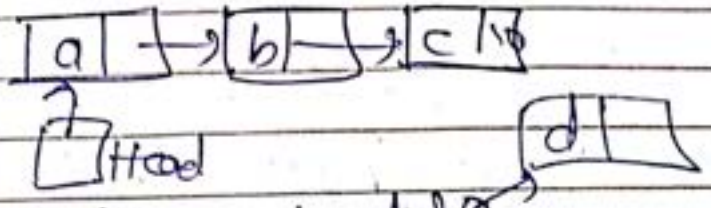
or `temp=stack[top--];`

In both array & LL push() & pop() are O(1)

(ii) Linked list

```
struct node {
  int i;
  struct node *link; // LL
};
```

Let's say



and we want to push d so head → d and d → link → a, so that pushed element always remain at top.

```
void push(int item) {
  struct node *p = malloc(sizeof(struct node));   // or (struct node*) malloc
  if (p == NULL) { printf("error of malloc"); }
  p → data = item;
  p → link = NULL;
  p → link = head;
  head = p;
}

int pop() {
  int item; struct node *p;
  if (head == NULL) { printf("underflow"); return -1; }
  item = head → data; p = head;
  head = head → next; free(p); p → link = NULL;
  return item;
}
```

Scanned with CamScanner

2 pointers front & rear

Queue (FIFO/LILO)

→ Implementation using array

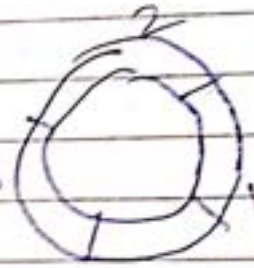
Stack [ ] → Inserting way (No problem)
↳ Deleting way

queue [ ] → Inserting way
→ Deleting way

→ Problem: free spaces will be available at front & we won't be able to use them;

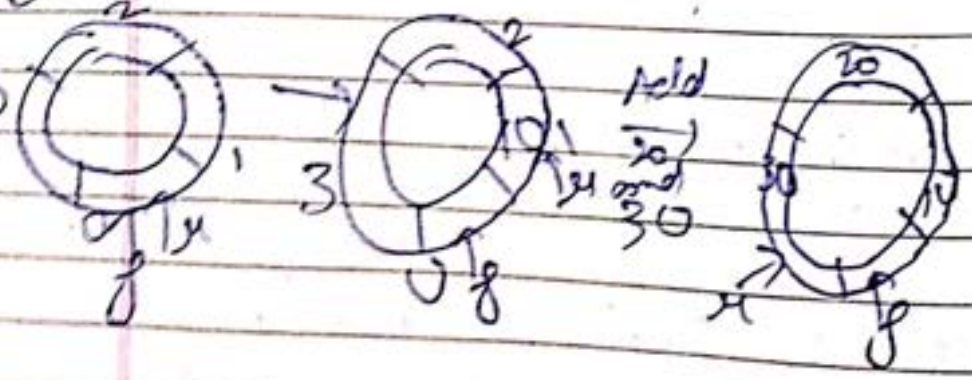
So above implementation is generally used, So we use

circular representation of array



→ Suppose if n spaces, we will use n-1, so that we will be able to know where is the beginning & where is end, etc.

Eg → Increment rear then insert



So a
we
here
f
Cre

If we
m

If
Vo
y

Scanned with CamScanner

So when we increment rear pointer we will use modular incrementing

here mod 4, so it comes to 0 then f=0 (initially), so it indicate array is full

(rear+1)%f (full)

If we delete element although f++, so more space will become available.

If is increment f & delete

```
void enqueue(item){
rear=(rear+1)%n;
if(front==rear){
printf("0 is full");
if(rear==0) rear=n-1;
else rear=rear=rear-1;
return;
}else{
q[rear]=item;
return;
}
}
```

Scanned with CamScanner

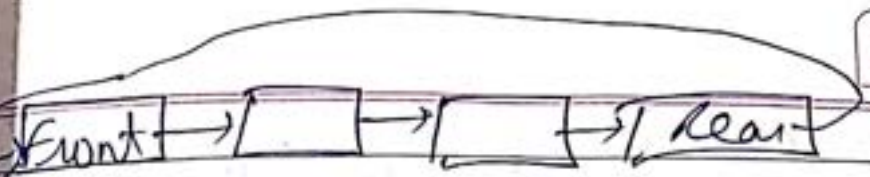
```
int dequeue(){
if(front==rear){
printf("Queue empty");
return -1;
}
else{
front=(front+1)%maxn;
item=q[front];
return item;
}}
```

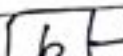
Gate 2012 Q. Suppose a circular queue of capacity (n-1) elements is implemented with an array of 'n' elements. Assume that the insertion & deletion operations are carried out using 'REAR' & 'FRONT' as array index variables respectively. Initially, REAR=FRONT=0. The conditions to detect queue full & queue empty are:

Ans (REAR+1) mod n = front
REAR = front

Gate 2004 Q. A circulating queue linked list is used to represent a queue. A single variable 'p' is used to access the queue. To which node should 'p' point to such that both the operations enqueue & dequeue can be performed in constant time?

Scanned with CamScanner

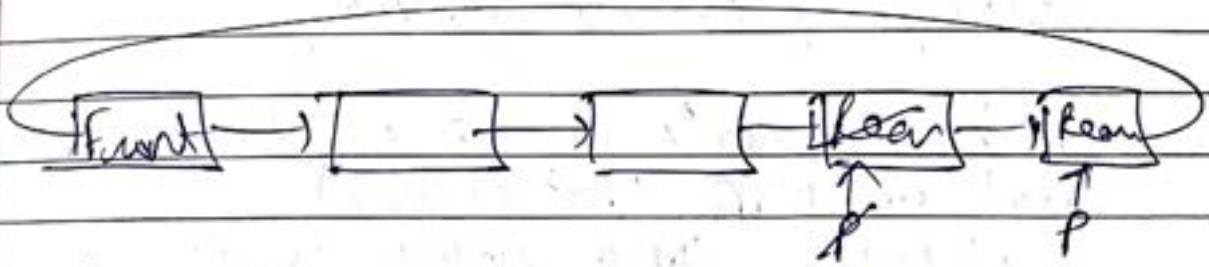




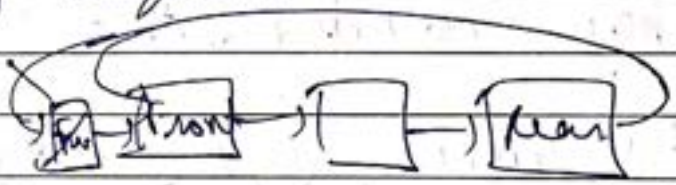
a) rear node b) front node c) not possible
d) node next to front

Ans a)

∵ if rear node enqueue using



If dequeue



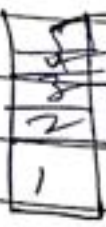
If front not possible ∵ for enqueue we have to traverse whole list (one-way

1994

Q-1 Which permutations can be obt in O/P (in same order) using a stack assuming that I/P is sequence 1 2 3 4 5 in that order?

a) 3, 4, 5, 1, 2 b) 3, 4, 5, 2, 1 c) 1, 5, 2, 3, 4 d) 5, 4, 3, 1, 2

a)



3 4 5 2 1 b) correct

Not possible

Scanned with CamScanner

Priority Queue → There are keys and values corresponding to it and they have priority when you want to delete a key this operation will depend on priority (if min priority then deleted) Inserted in any order

Gate

How we can mimic stack operation using priority queue?

→ Push elements in any order but on pop that element would come outside which is pushed last (least priority. i.e least key associated with it

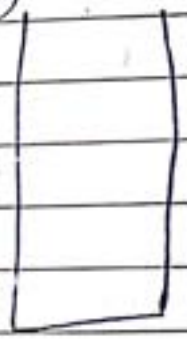
Gate 2011 Q: A priority queue 'Q' is used to implement a stack 'S' that stores characters. PUSH(c) is implemented as Insert(Q, c, k) where k is an appropriate key chosen by the implementation. If Q is implemented as DELETEMIN for a sequence of PUSH operations, the keys are in

a) Non-decreasing order
b) Non-increasing order
c) Strictly increasing order
d) Strictly decreasing order

Scanned with CamScanner

Gate 2003

Q-1 Let 'S' be a stack of size $n \geq 1$. Starting with the empty stack, suppose we push the first 'n' natural numbers in sequence & then perform 'n' pop operations. Assume that push & pop operations take 'x' seconds each, and 'y' seconds elapse b/w the end of one such stack operation and the start of the next operation. For $m \geq 1$, define the stack-life of 'm' as the elapsed from end of push(m) to the start of pop operation that removes 'm' from S. The average stack-life of an element of this stack is

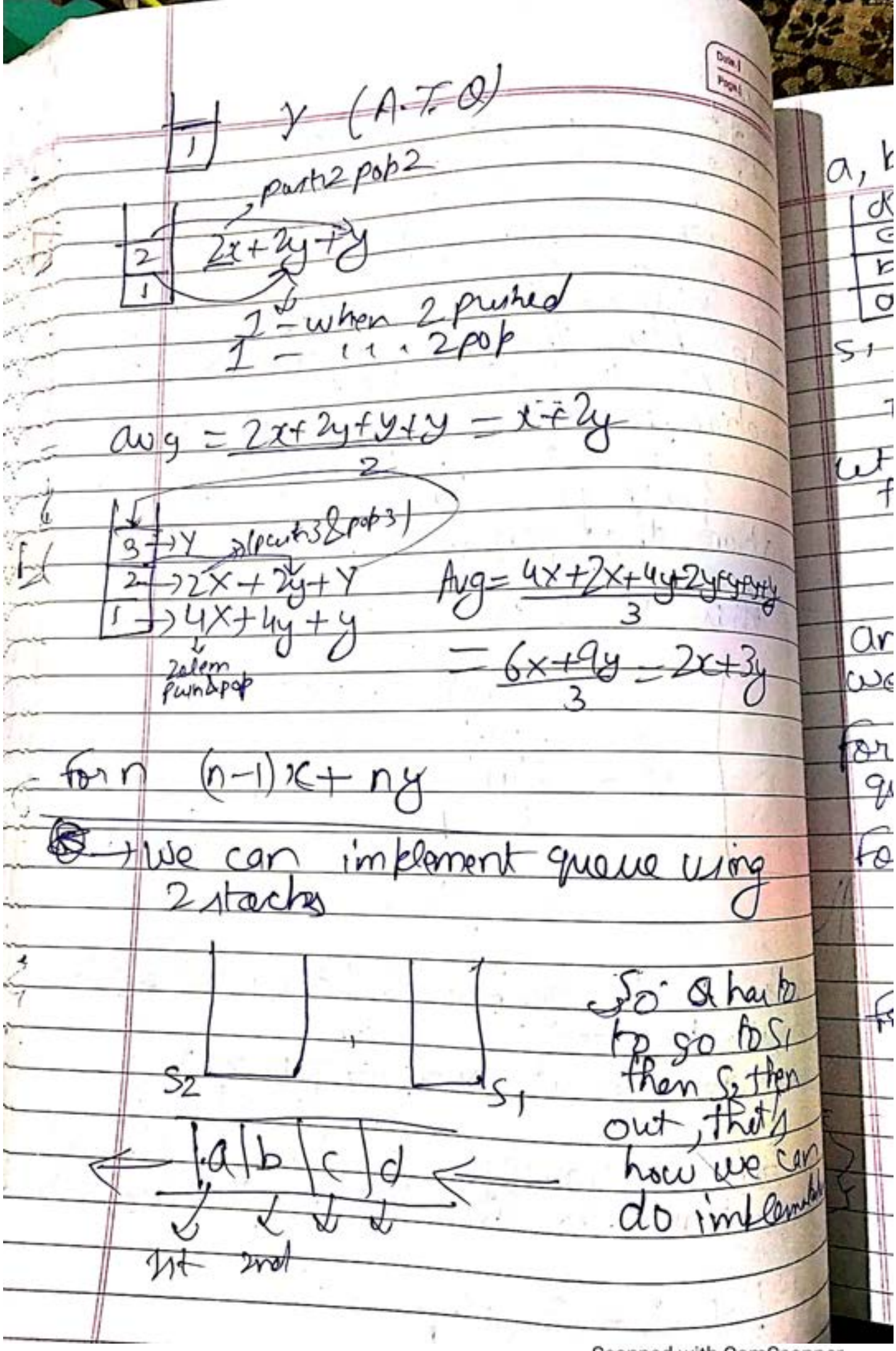


push → x
pop → x
b/w push & pop → y
" pop & " → y
" pop & push → y
" push & " → y

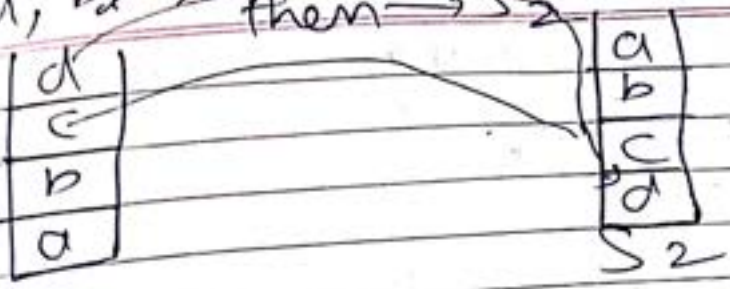
Suppose stack empty & 1st pushed then time start → x
then we pop it so x + y more time
So total → 2x + y → lifetime of 1st element
And we have to find lifetime average so

Upper bound of $\frac{1 + 2 + 3 + \cdots + n}{n}$

Scanned with CamScanner


1 → y (A.T.O)
push2 pop2
2
1
2x+2y+y
I - when 2 pushed
I - " " 2pop
avg = 2x+2y+y+y / 2 = x+2y
3 → Y (push3 & pop3)
2 → 2X + 2y + Y
1 → 4X + 4y + y
2 elem push & pop
Avg = 4x+2x+4y+2y+y+y / 3
= 6x+9y / 3 = 2x+3y
for n (n-1)x + ny
→ we can implement queue using 2 stacks
S2
S1
So a has to go to S1 then S2 then out, that's how we can do implement
a b c d
1st 2nd

a, b, c, d → S_1 then → S_2



pop d from S_1 put to S_2 etc. etc.

S_1 then when you pop from S_2 then a, b, c, d

let's say new element e f g comes then we push them to S_1 but don't pop it to S_2 until & unless S_2 is emply.

and also whenever we shift from S_1 to S_2 we always pop all elements

for one element a to come from queue (made of 2 stacks) a → 2push. → 2pop

for a, b → 2push
2pop
2push
2pop

for n variables 2npush
2npop

Scanned with CamScanner

Gate 2006

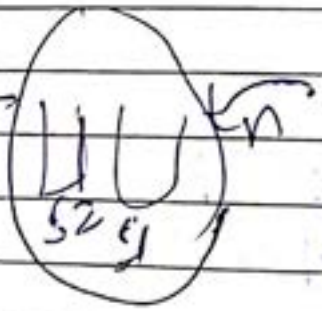
Q: Q is implemented using 2 stacks S_1 & S_2 as shown:

```
void insert (Q, x){
  push(S1, x);
}

void delete (Q){
if (stack-empty(S2)) then
if (stack-empty(S1)) then{
    printf ("Q is empty");
    return;
}
else while (!stack-empty(S1)){
    x = pop(S1);
    push(S2, x);
}
pop(S2);
}
```

If we do n-inserts & m(≤n) deletes then we require X-pushes & Y-pops find relation b/w X, Y, m, n?

Ans



~~0 pushes~~

Worst case: 1 element

Push: $2m + (n-m)$

$= m+n$

Let's say m elements came out

Page 141

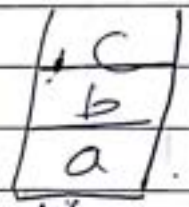
Worst Case:
2n pushes

Pop Best Case: 2m
Worst Case = O(n+m)

$n+n \leq X \leq 2n$
$2m \leq Y \leq n+m$

Gate 2000
Q) Suppose a stack implementation supports, in addition to PUSH and POP, an operation reverse that reverses the order of elements on the stack.
- Implement a queue using the above stack implementation, show how to implement "ENQUEUE" using a single operation & "DEQUEUE" using a sequence of 3 operation

→ Enqueue

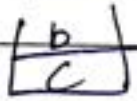


abc

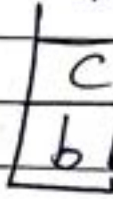
dequeue → reverse



pop



again reverse it.

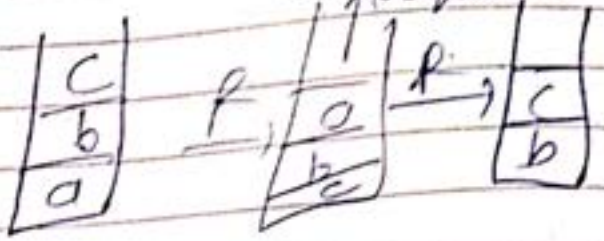


so that we can push more elements

Scanned with CamScanner

Other way Enqueue - 3
dequeue - 1

Enqueue



Infix to postfix conversion algorithm

Eg → a+b → infix expression
→ ab+ → postfix
a-b*c → infix
→ a+bc* (∵ * high precedence) → abc*+
→ ab+c- ∵ operator read

if a+b-c →

2 imp questions
Q which DS for?
A → Stack
Q → what pushed
A → Only operators (not operands)

eg :- a+(b-c)
a+bc-
abc-+ Ans

→ a+b*c
a+bc*
abc*+
simple Stack

Scanned with CamScanner

Date:
Page: 143

a + b * c

+	*	

a - print it
+ → 2 things
1 If empty push rightaway
2 Push only when inside one lower precedence
b → print it
* → Put it (high precedence than +)
c → print c

Now abc
After that pop all elements of stack & write
So abc*+

ii) a+b+c

*	+

a — a print // a
* → * stack
b → b print // ab
+ → to push star we pop out stack till + precedence higher or stack empty, so after popping * // ab*
c → ab*c
→ ab*c+

Scanned with CamScanner

1 2 3 4 5 6 7 8 9

a + b + (c - d)

TOP = -1

Precedence
P.S + < C
C < +

1) a
2) a
3) ab
4) ab*
5) ab*
6) ab+c
7) ab+c
8) ab+cd

whenever ')' i.e closed parentheses, we don't do with it anything neither Print nor push

When) keeping on popping the stack until you get C, so

9) ab*cd-
10) ab*cd-+

Algorithm:
1) Create a stack (≤ n size)
2) For each character 't' in I/P

{ if ('t' is operand)
output 't';
else if ('t' is Right Parathesis)
{ Pop & o/p tokens until a left parenthesis is popped but don't o/p Left Parenthesis}
}
else // t is an operator or left parenthesis
{
Pop & o/p tokens until one of lower priority than 't' is encountered or a Left parenthesis is encountered or the stack is empty
}

6) Pop & o/p all tokens until the stack is empty.

Algorithm for postfix evaluation → eg: to evaluate postfix expression we have to scan it from left to right & then come back to it
& Time & space complexity O(n)

1) Scan the postfix string from left to right
2) Initialize an empty stack.
3) Repeat steps 4 & 5 till all the characters are scanned.
4) If the scanned character is an operand, push it onto the stack
5) If the scanned character is an operator, and if the operator is unary then pop an element from the stack. If the operation is binary, then pop 2 elements from the stack. After the popping the elements, apply the operator to those popped elements. Push the result onto the stack
6) After all elements are scanned, the result will be in the stack

eg 3+2*1 → Postfix → 32+1*+

3 | 2 | 1 → 3 | 2 → 3+2=5 → 5 Ans

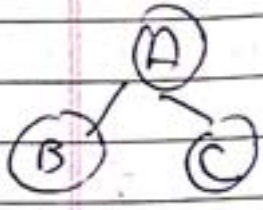
* binary to so pop 2 & 1 & 2*1=2
1st pop (2nd operand) 2nd pop (1st operand)

→ known as operand stack

Scanned with CamScanner

Tree

Traversing — Inorder — L Root R — BAC
— Preorder — Root L R — ABC
— Postorder — L R Root — BCA

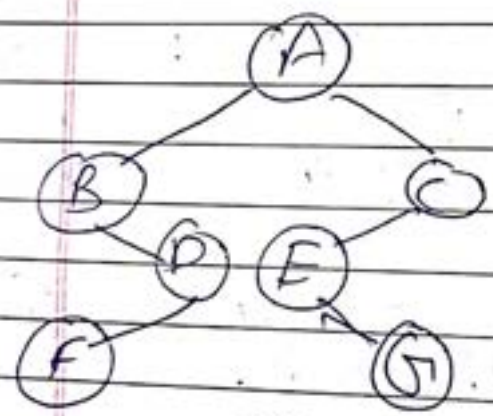


~~Preorder~~

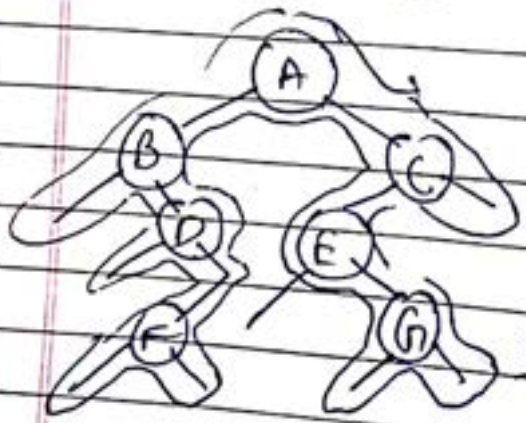


ABC — Print 1st Preorder time
BAC Inorder — Print 2nd time
BCA → 3rd time Postorder.

Eg



Ans



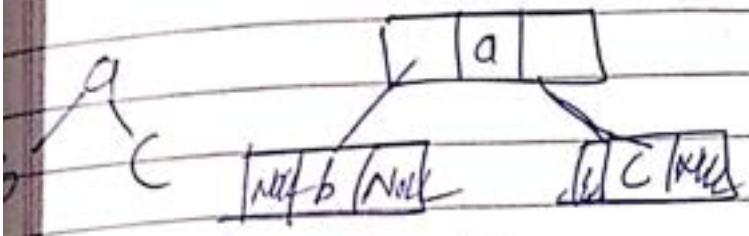
Print 1st time ABDFCEG
Pre
Inorder BFDAEGC
Inorder BFDAEGC

Post FDBGECA

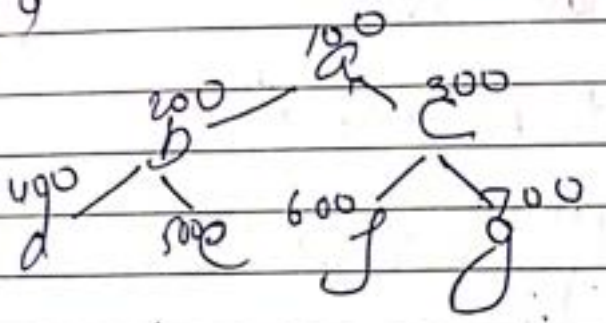
Scanned with CamScanner

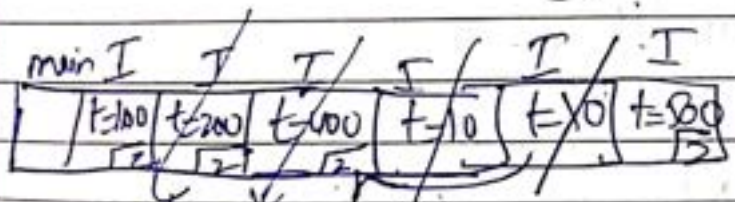
Inorder → L Root R

```
struct node{
   int data;
   struct node *left;
   struct node *right; };
```




```
void inorder(struct node *t) {
   if (t != NULL)
   { 1 Inorder (t->left);
     2 printf ("%d", t->data);
     3 Inorder (t->right);
   }
}
```





O(n) ~~O(n)~~ O(3n)

Space complexity O(n)

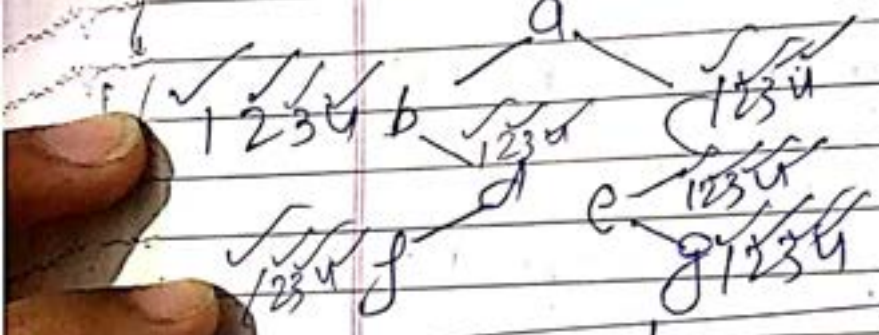
Scanned with CamScanner

Double order traversal

```
Do(t){
if(t){
1 printf(t->data);
2 Do(t->left);
3 printf(t->data);
4 Do(t->right);
}
```

1 2 3 4

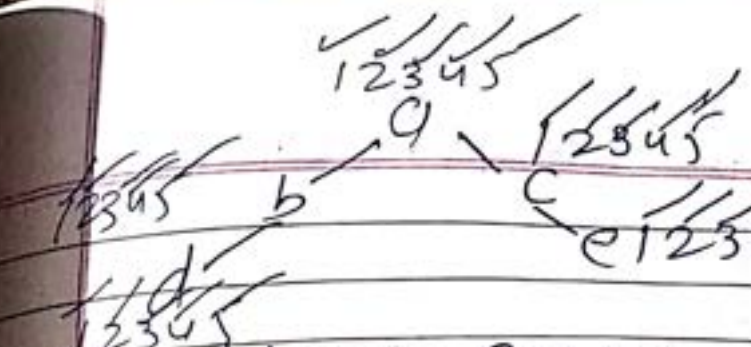
abbdffdaceegc



Triple order traversal

```
void To(struct node *t){
if(t){
1 printf("%d", t->data);
2 To(t->L);
3 Printf("%d", t->data);
4 To(t->R);
5 printf("%d", t->data);
}
}
```

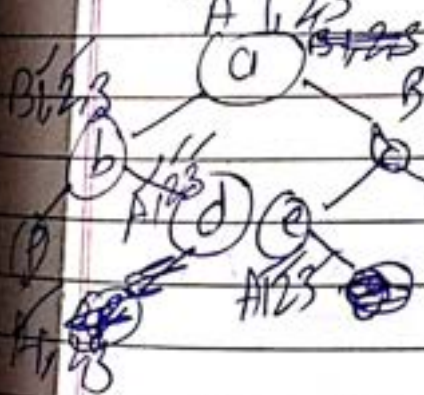
Scanned with CamScanner



abdddbbaCC eee ca

Indirect recursion on trees (A→B→A)

```
void A (struct node *t) {
  If (t) {
  1 printf ("%d", t->data);
  2 B(t->left);
  3 B(t->right);
  }}
void B(struct node *t) {
  if (t) {
  1 A(t->left);
  2 printf("%d", t->data);
  3 A(t->right);
  }}
```



A called first

agbdeCg

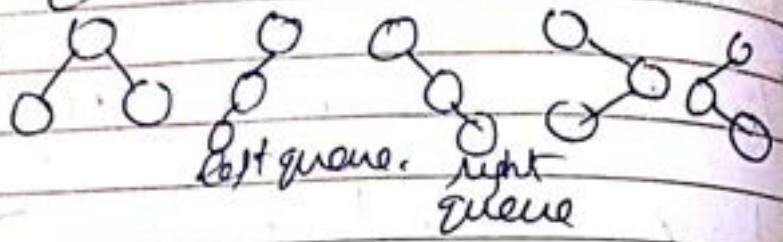
B called first bfdaCg

Q→ How many trees are possible with n nodes

1 node → 1

2 node → 2

3 node → 5



General $\frac{^{2n}C_n}{n+1}$ (# of binary trees (unlabelled nodes)

What if labelled nodes



think like

$\frac{}{3}$ $\frac{}{2}$ $\frac{}{1}$

→ // → //

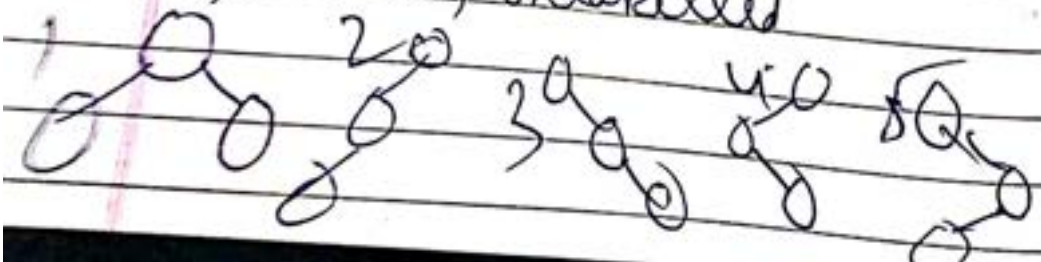
$3! = 6$

General → $n! \times \frac{^{2n}C_n}{n+1}$

All the binary trees with 3 nodes A, B, C which have preorder ABC

Ans Total = $5 \times 3! = 30$

Let's solve by unlabelled

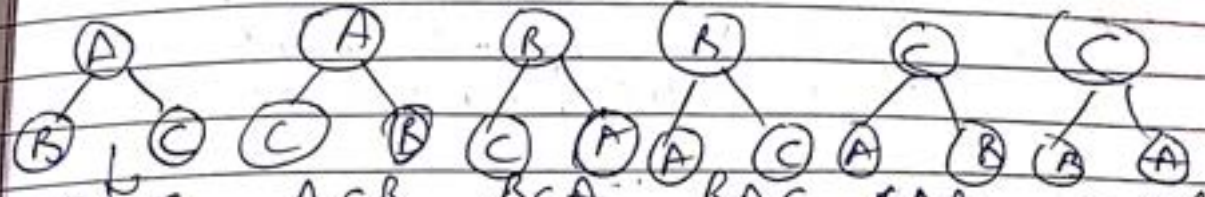


Scanned with CamScanner

P.S 1, 2, 3, 4, 5, 6

Page 151

and only 1 out of these 6 i.e will have ABC (∵ everytime diff traversal)

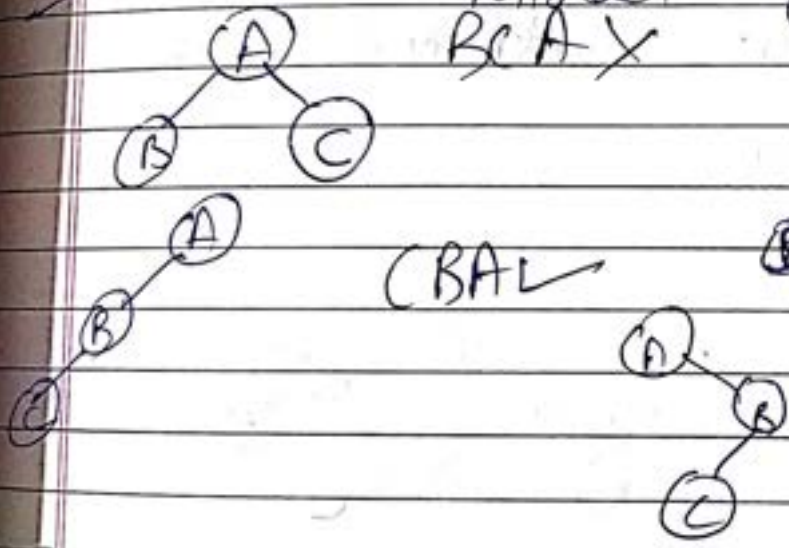


ABC ACB BCA BAC CAB CBA

Ily for others Ans → 5 $\left(\frac{2n C_n}{n+1}\right)$

Q Same & Preorder ABC && Postorder CBA

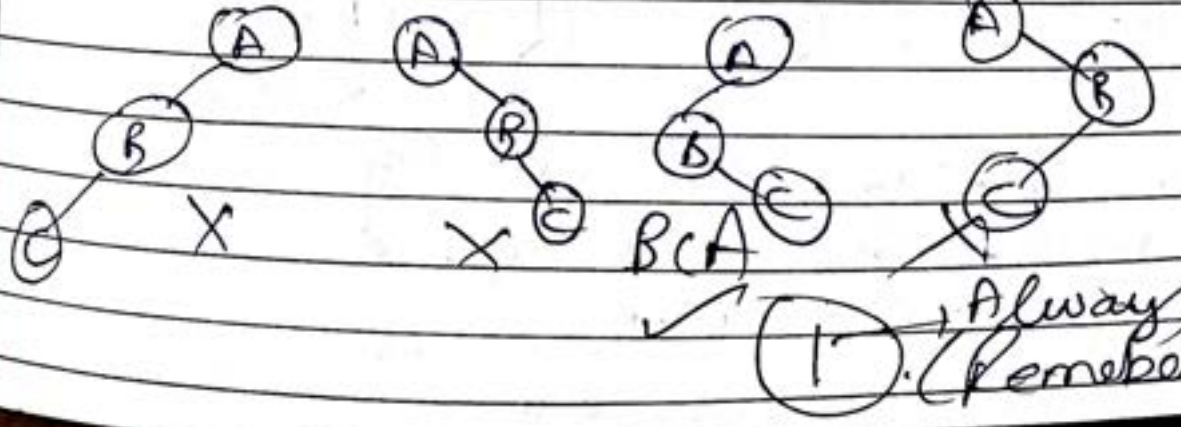
Postorder BCA ✗



CBA ✓ CBA ✓ CBA ✓ (4)

Q Preorder ABC && Postorder CBA & inorder BCA

Ans Take above 4



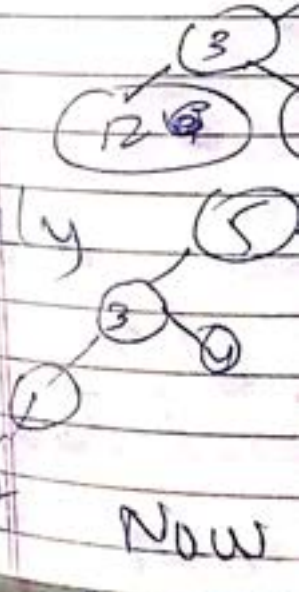
X X BCA ✓

(1) Always (Remember it)

Scanned with CamScanner

n nodes





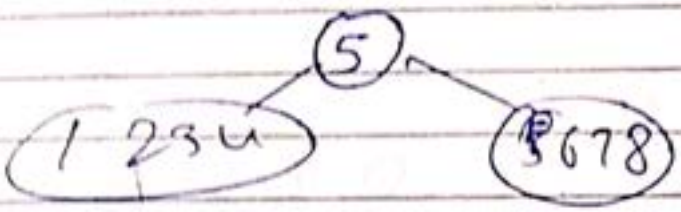
Scanned with CamScanner

So we see inorder
i.e. ABC → we choose A
BAC → we can easily guess A is root
L Root R

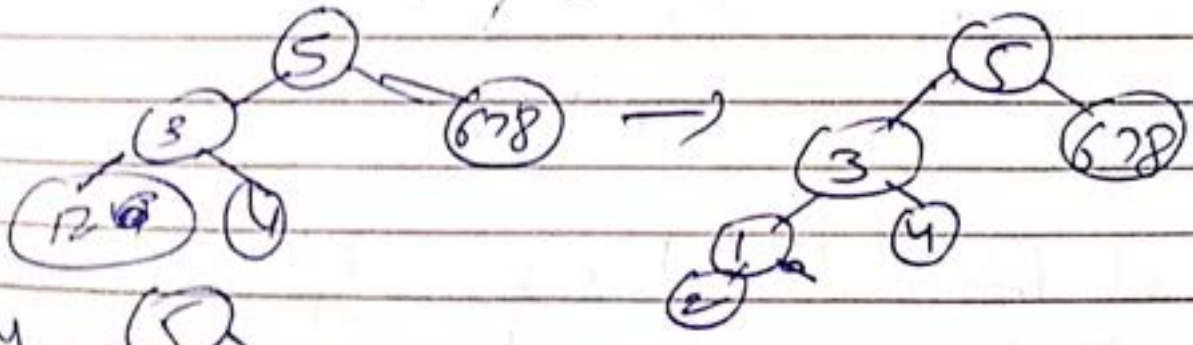
So B A C i.e A then A
Always root B C
B is first Pre → (A) if A doesn't fit we will see choose B & so on

Ex: In: 1, 2, 3, 4, 5, 6, 7, 8
Pre: 5, 3, 1, 2, 4, 6, 8, 7
Post: ? ? ? ? ? ? ? ?

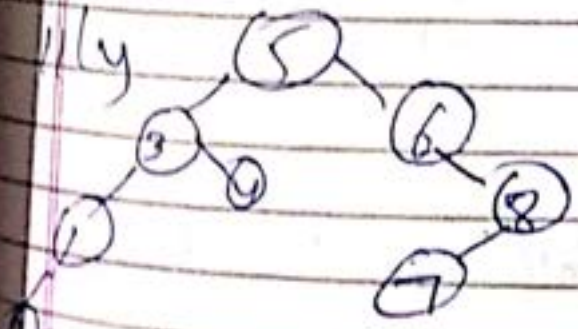
Try 5 root Pre: 5, 3 so



Now in 1, 2, 3, 4, 3 is root ∴ Pre = 5, 3 etc.



Ily

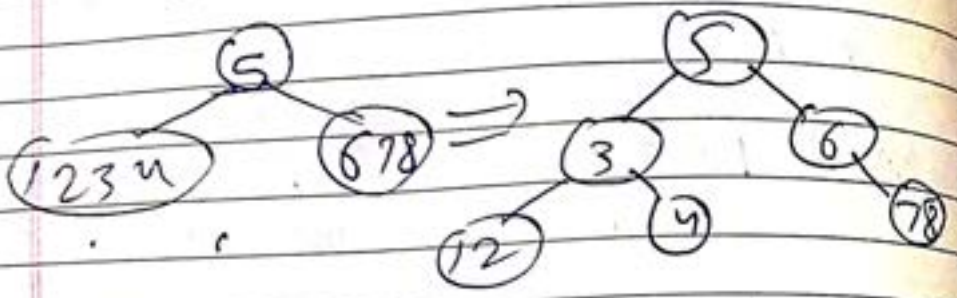


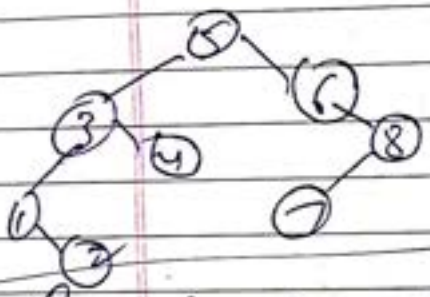
Now Post: 2 1 4 3 7 8 6 5

Scanned with CamScanner

Now reverse engineer some menu set

IN: 1, 2, 3, 4, 5, 6, 7, 8
Post: 2 1 4 3 7 8 6 5 → Root





Recursive prog to count # of leaves

If leaf NL(T) = 1, T is leaf
else NL(T) = NL(LST) + NL(RST)

leaf if | 10 | 11 |
else not leaf

```
Sg int NL(struct node *t) {
  if (t == NULL) return 0;
  else if (t->left == NULL && t->right == NULL)
    return 1;
  else
    return (NL(t->left) + NL(t->right));
}
```

Let's say

```
if NN L(T) =
else = 1
int NN
  if(
  if(
  else
    ret
}
```

Time
Space

23/9/20
full

Scanned with CamScanner

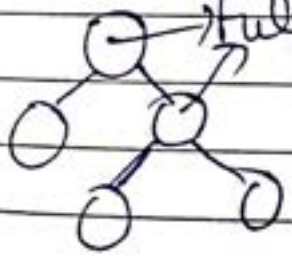
Let's say count # of non-leaves

If NNL(t) = 0 ⇒ T is leaf
else = 1 + NL(LST) + NL(RST)

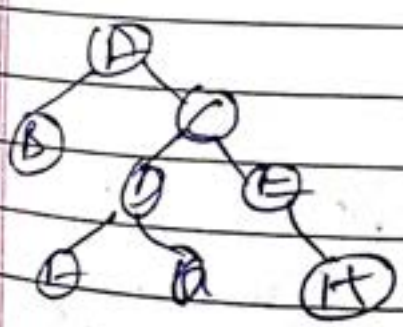
```
int NNL(struct node *t){
    if(t==NULL) return 0;
    if(t->left==NULL && t->right==NULL)
        return 0;
    else
        return(1+NNL(t->left)+NNL(t->right));
}
```

Time complexity O(n)
Space complexity O(n) — Always implicit stack in recursion

23/9/20
full node - A node having 2 children (not 1)
→ fullnodes(2 here).



full node ⊆ Non leave



L : B, F, G, H
NL : A, C, D, E
FN : A, C, D

Scanned with CamScanner

LST → Left Sub Tree

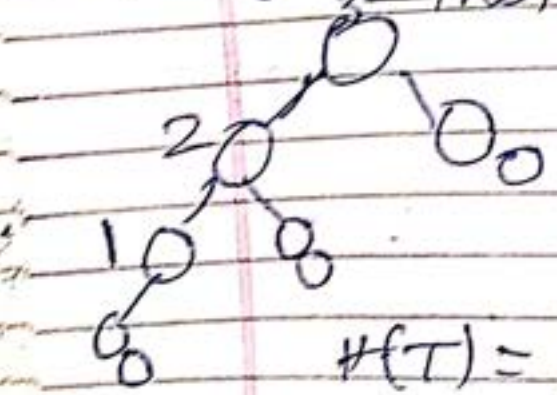
F(N) = 0; T = NULL
= 0; T is a leaf
= FN(T→LST) + FN(T→RST) + 1; 2 children

```
int FN(struct node *t){
    if(!t) return 0;
    if(!t->left && !t->right)
        return 0;
    return FN(t->left) + FN(t->right) +
        (t->left && t->right) ? 1 : 0;
}
```

Time complexity = O(n)

Recursive program to find height of a tree

→ Height of node = max(length of path to it from leaf)
→ height of tree



H(T) = { 0; T is empty
0; T is a leaf
1 + max(H(LST), H(RST)); otherwise }

```
int H (struct node *t){
  if(!t) return 0;
  if(!t->left && !t->right) return 0;
  int l,r;
1 l= H(t->left);
2 r= H(t->right);
3 return (1+((l>r)?l:r));
4 }
```

1,2,3

1,2,3

1,2,3

0

Time complexity O(n)
Space O(n)

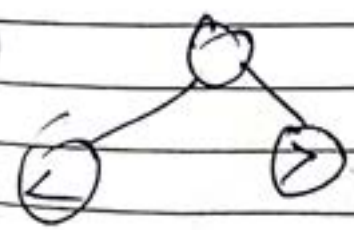
Binary Tree Disadv → No proper pattern for searching a node :— → O(n)
∴ we have to visit every node in worst case

Sol^n → Binary search tree

O(log n) is balanced BST

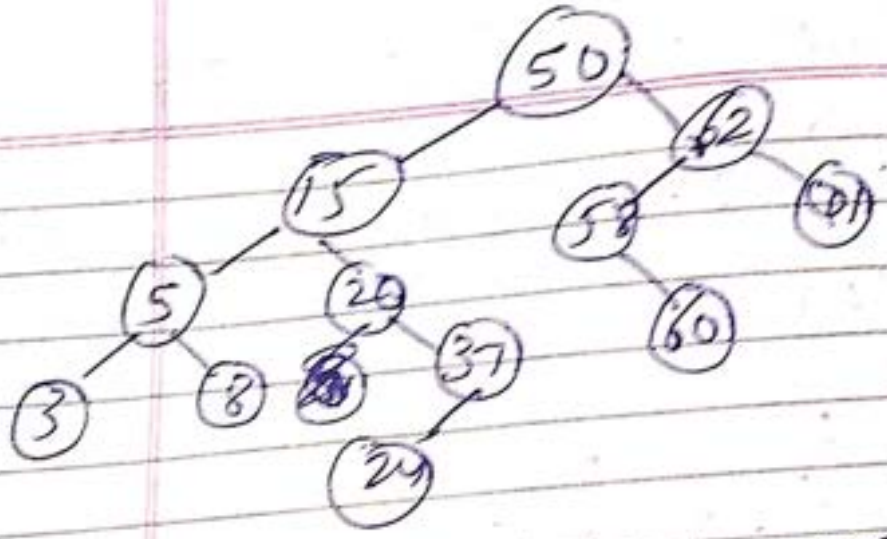
Code 1996 Insert 50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24 into a BST

General



1) Nodes are not ordered and
2) left have smaller right have larger values of node

Scanned with CamScanner



Property - 1) If a tree is a BST then any subtree is also a BST

2) Inorder traversal gives elements in increasing order (very Imp)

So Inorder of above

3 5 8 15 20 29 37 50 58 60 62 91

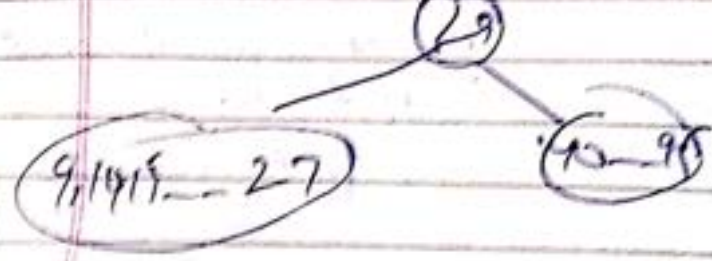
Gate 2005 Q: Postorder: 10, 9, 23, 22, 27, 25, 15, 50, 95, 60, 40, 29

Find Inorder & Preorder

Ans → You already know Inorder

Inorder = 9, 10, 15, 22, 23, 25, 27, 29, 40, 50, 60, 95

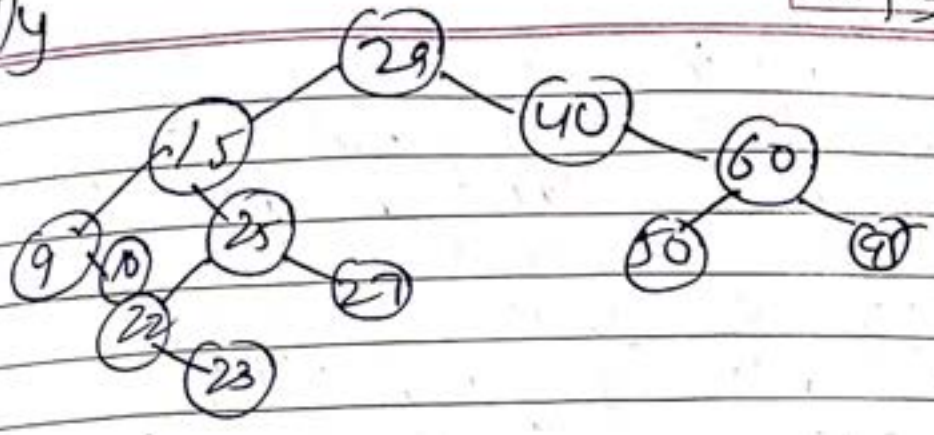
Now Root = 29 (You know why)



Scanned with CamScanner

Page 159

1/4



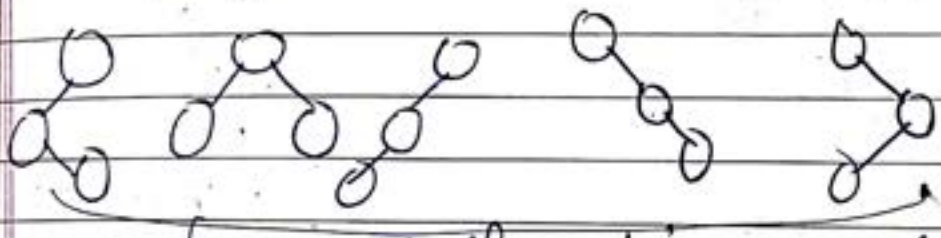
Preorder = 29 15 9 10 25 22 23 27 40 60 50 95

Code 2005

Q. How many BSTs are possible with 4 distinct keys

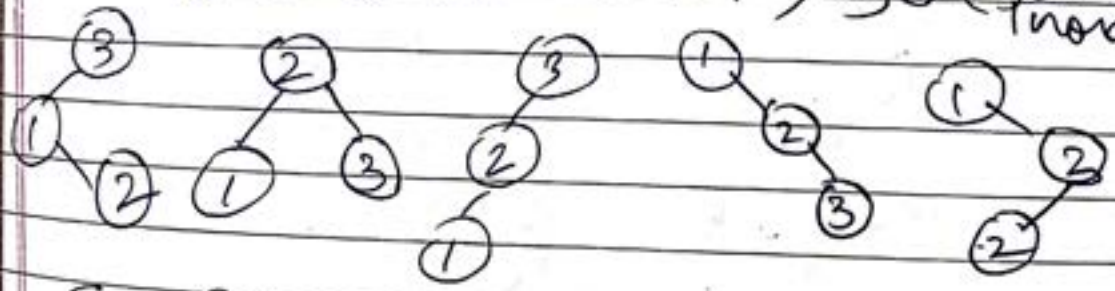
Ans → BST with unlabelled node - concept

Let's do for 3 distinct keys (1, 2, 3)



from all these binary only 1 BST possible with each combination

not 5×6 but 5×1, 10 (Just see Inorder already)



$S = \frac{^{2n}C_n}{n+1}$ for $n=3$

So for $n=4$ $\frac{^{8}C_4}{5} = \frac{8\times7\times6\times5}{5\times4\times3\times2\times1} = 14$ Ans

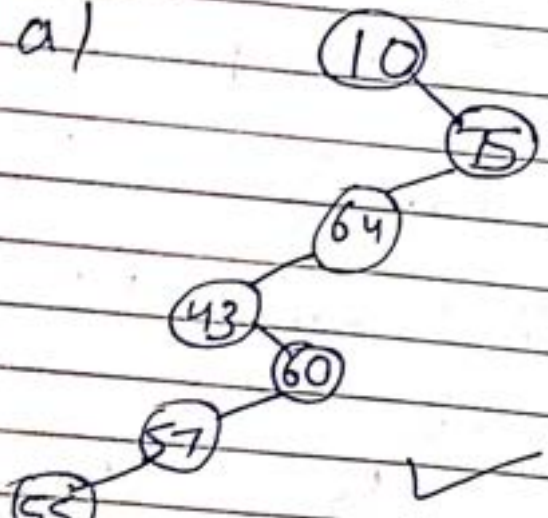
Scanned with CamScanner

Gate IT-2006

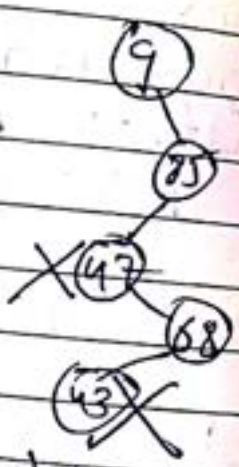
Q. (1-100) in BST
Search for 55. Which of the following sequences can't be the sequence of nodes examined.

a) {10, 75, 64, 43, 60, 57, 55}
b) {90, 12, 68, 34, 62, 45, 55}
c) {9, 85, 47, 68, 43, 57, 55}
d) {79, 14, 72, 56, 16, 53, 55}

Ans → a)

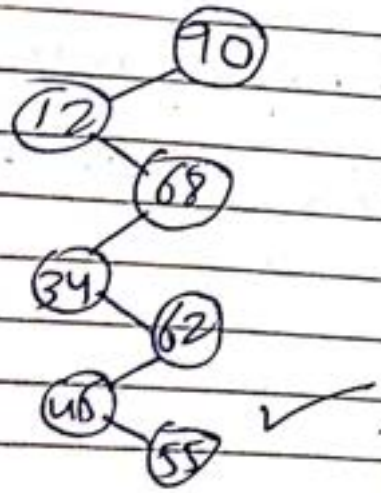


c)

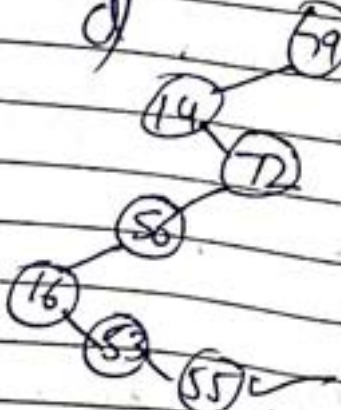


c) Ans
47 → Right → 43
So wrong

b)



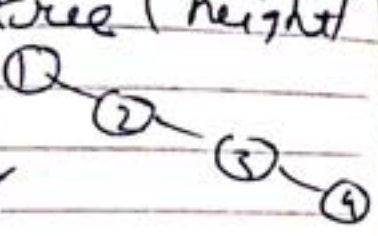
d)



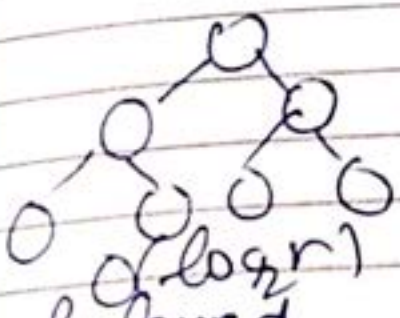
Scanned with CamScanner

Time to search in a binary ST
1 ——— n elements

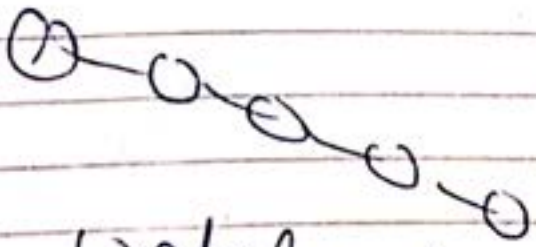
Ans: Depends on depth of tree (height of tree)
Worst Case O(n) if



So we make balanced BST



O(log n)
Balanced



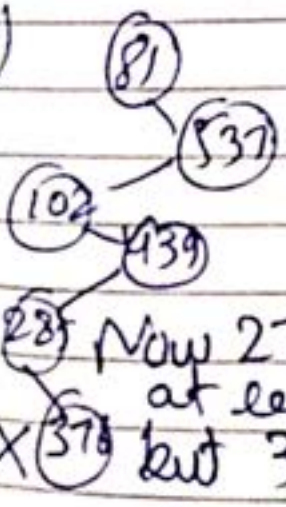
Unbalanced
O(n)

17 2008
Q: A BST stores value in range 37 to 573. Consider the following sequence of keys

I) 81, 537, 102, 439, 285, 376, 305
II) 52, 97, 121, 195, 242, 381, 472
III) 142, 248, 520, 386, 345, 270, 307
IV) 550, 149, 507, 395, 463, 402, 270

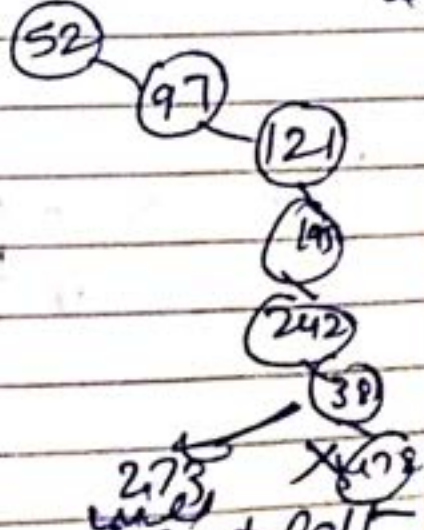
Now if 273 is inserted which is unsuccessful

From I)



Now 273 will be at left of 285 but 376 > 273 > 285
So unsuccessful

II)

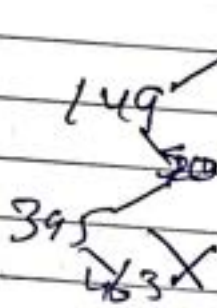


273 will be at left of 381 but 273 > 242
So unsuccessful

Scanned with CamScanner

III 142 → 248 → 520 → 386 → 345 → 270 → 307 ✓ possible.

IV

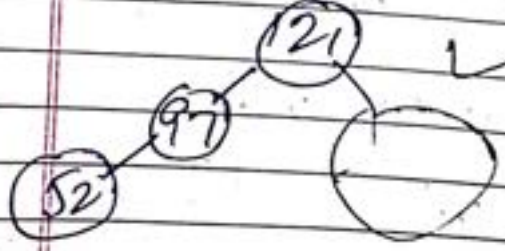


unsuccessful.

Option a) I, II, IV are inorder sequences of 3 different BSTs.

b) I is preorder sequence of some BST with 439 as the root

c) II is an inorder of some BST with 121 as root & 52 as leaf.



✓ Yes possible.

c) true

d) IV is a postorder sequence of some BST with 149 as root

149 ✗ ∵ 270 at last

Gate – 2008

I : MBCAFHHPYK
II : KAMCBYPFH
III : MABCKYFPH

Find In, Pre, Post out of I, II, III & make tree.

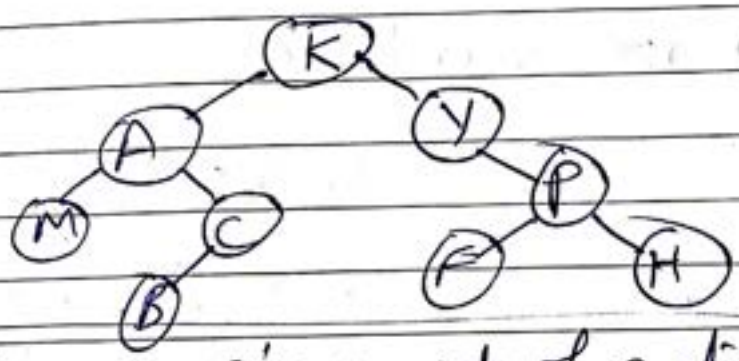
Ans Hint If Pre → 1st Node Root
Post → last Node Root

let's write end & last

I	M	K
II	K	H
III	M	H

So III is inorder traversal for we

also II is pre & I is post



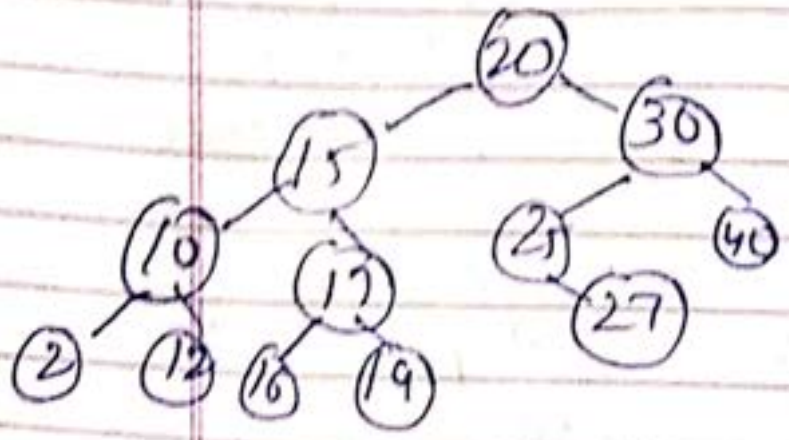
Gate 2011
Q-1 We are given set of n distinct elements & an unlabeled tree with 'n' nodes. In how many ways we can label the tree with the given set so that it becomes a BST

a) 0 b) 1 c) n! d) $\frac{2^n C_n}{(n+1)}$

Ans b) : we are already given a tree. We don't have to design it ourselves. If we have to do then d) Ans.

Scanned with CamScanner

Deleting a node from BST



(i) leaf
(ii) non-leaf
a) one child
b) two children

(i) leaf → directly delete :: O(1)
no need to worry about anything

(ii) Non leaf a) one child

Let's delete node (25)

So if we directly delete (27) 40 goes out

So we connect (27) to (30) & delete (25)

b) two child

Algo I → go to right child & place least element, eg if we delete (15) then (16) put in place of (15) & rest same.

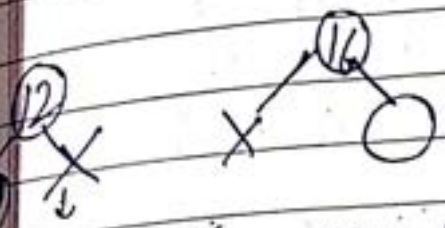
Algo II → go to left child & place highest element, eg (15) to delete, take (2) & replace with (15)

In all operations height either stay or remains constant

Scanned with CamScanner

Date
Page 165

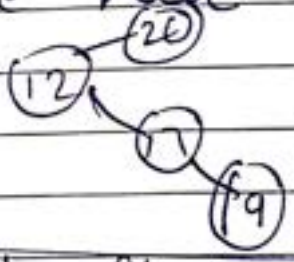
Now in case (i) b)
either 12 & 16 are leaf or they can have 1 child



∴ 12 is greatest it can't have right child
& 16 is smallest " " " left "

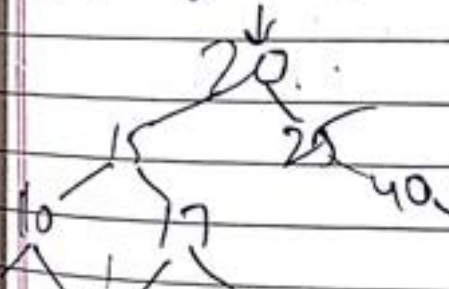
Inorder succesor - least element in right subtree $= 16$
Inorder predecesor - greatest " " left "
$= 12$

If we have



and we want to delete 12, then entire link is connected to 20 not 17 alone

Now to find inorder succesor & inorder predecesor we need to know how to find smallest & largest element in a tree given its root.



point-er to 20, we want to find least & max element

```
find_inord_pre (struct node *t)
{ while (t->left)
    t = t->left;
find_inord_succ (struct node *t) {
while (t->right)
  t = t->right;
}
```

Scanned with CamScanner

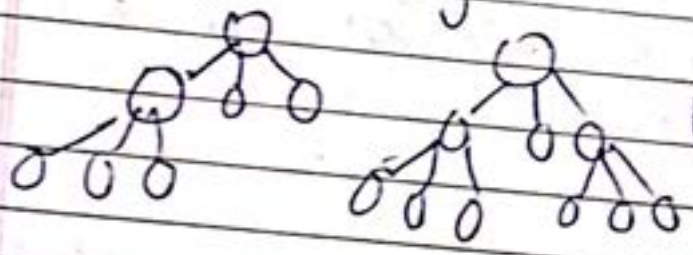
Gate 1998

Q→ A complete n-ary tree is one in which every node has 0 or n children. If 'x' is the # of internal nodes of a complete n-ary tree, the number of leaves in it is given by

a) $x(n-1)+1$ b) x^{n-1} c) $x^{n}+1$ d) $x(n+1)$

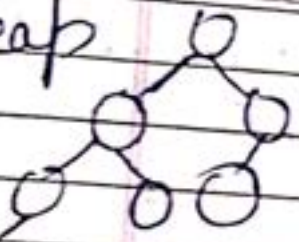
Ans

comp 3-ary



etc, etc.

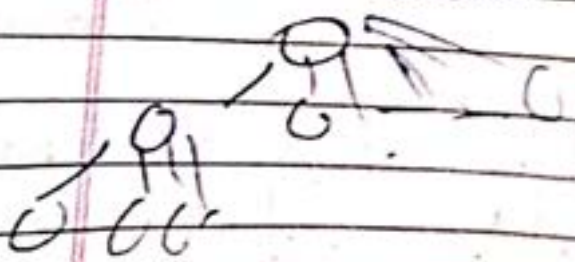
Heap



✗ not allowed until is filled.

1 internal node – n leaves

2 int nodes – (n-1) + n leaves
= 2n-1



3 internal node – (2n-2) + n
= 3n-2

(2) Correct

Gate 2002

Q. The # of leaf nodes in a rooted binary tree of 'n' nodes, with each node having 0 or 3 children is

a) n/2 b) (n-1)/3 c) (n-1)/2 d) (2n+1)/3

Ans-1 Basically extension of previous question

Using formula $L = x(3-1)+1$ (L = leaves)

$L = 2x+1$

x - Internal nodes

also $L + x = n$

$2x - L = -1$

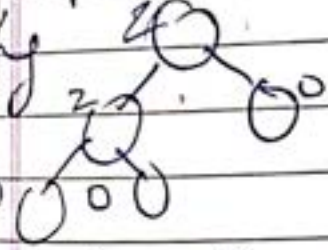
$x + L = n$

$3x = n-1$, $x = (n-1)/3$

So $L = n - x = n - \left(\frac{n-1}{3}\right) = \frac{3n-n+1}{3}$

$= \frac{2n+1}{3}$ (d)

Complete binary tree - 0 or 2 children



For strict binary tree

Full binary tree



Recursive prog to check complete binary tree

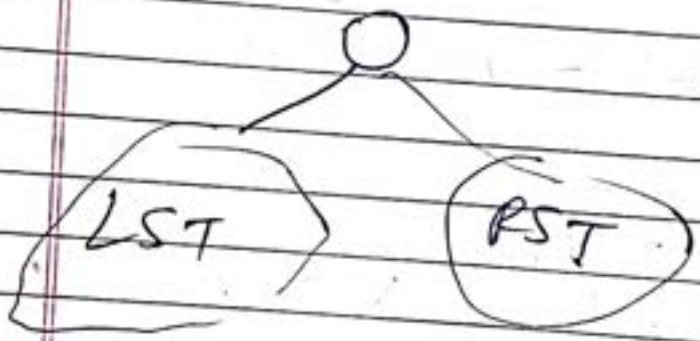
```
int isComplete (struct node *t){
    if (t == NULL) return 1;
    if (t->left && !t->right) return 1;
else if (t->left && t->right
return (isComplete(t->left) && isComplete(t->right));
```

Scanned with CamScanner

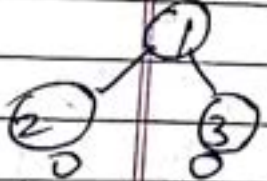
Motto → In LL & array search time is O(n)
So trees → O(log n), but till now that's not the case, so

Introduction to AVL trees & balancing

AVL → If n nodes height always O(log n)
same after 3 insert
red black tree
11 (y (R-B) (2-3)
but we are studying AVL for time being

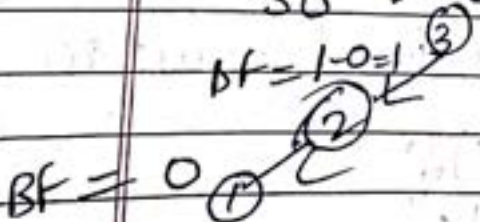


Balance factor, BF = Height(LST) − Height(RST)
if >= 2 or <= -2 then unbalanced



BF = 0
In most cases height starting from 0, but here for simplicity we assume

So BF = 2-0 = 2 unbalanced



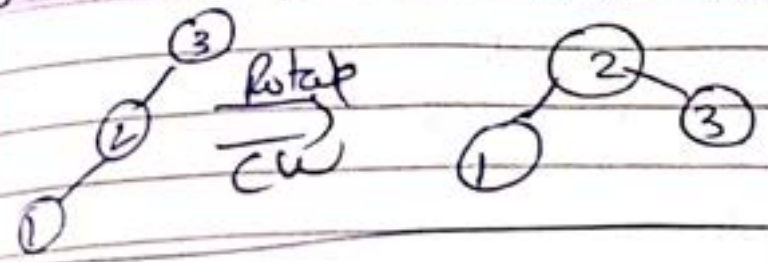
LL imbalance



Scanned with CamScanner

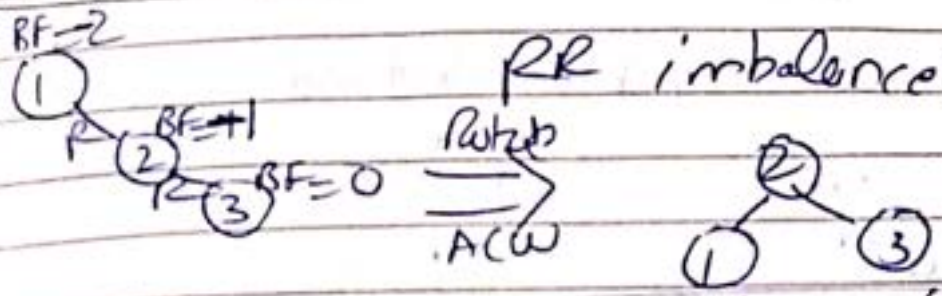
, 1 case is last (rotate with 2 childs)
we will see in eg.

Whenever LL imbalance rotate clockwise (with 2 as root)

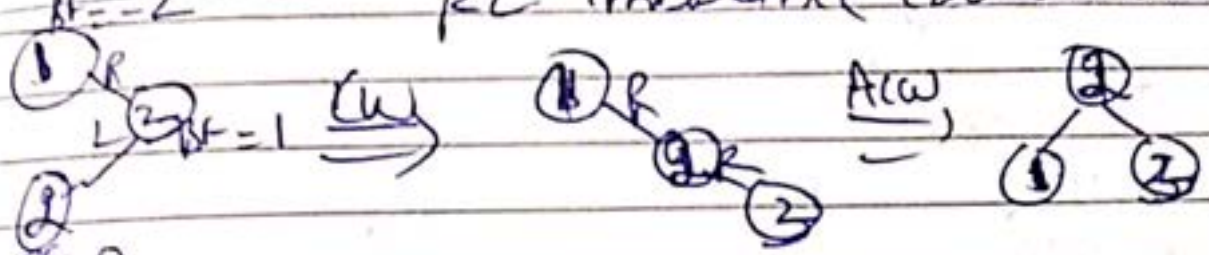


By practice.

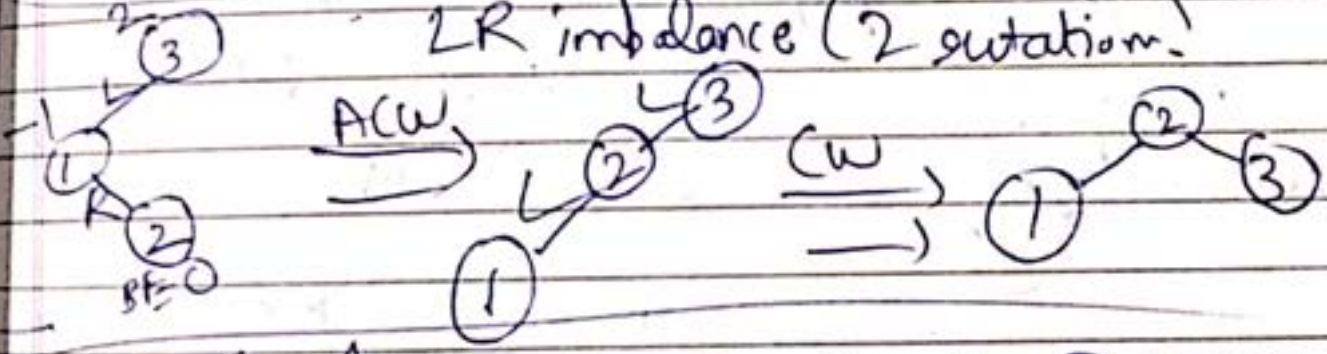
RR imbalance (1 anticlockwise)



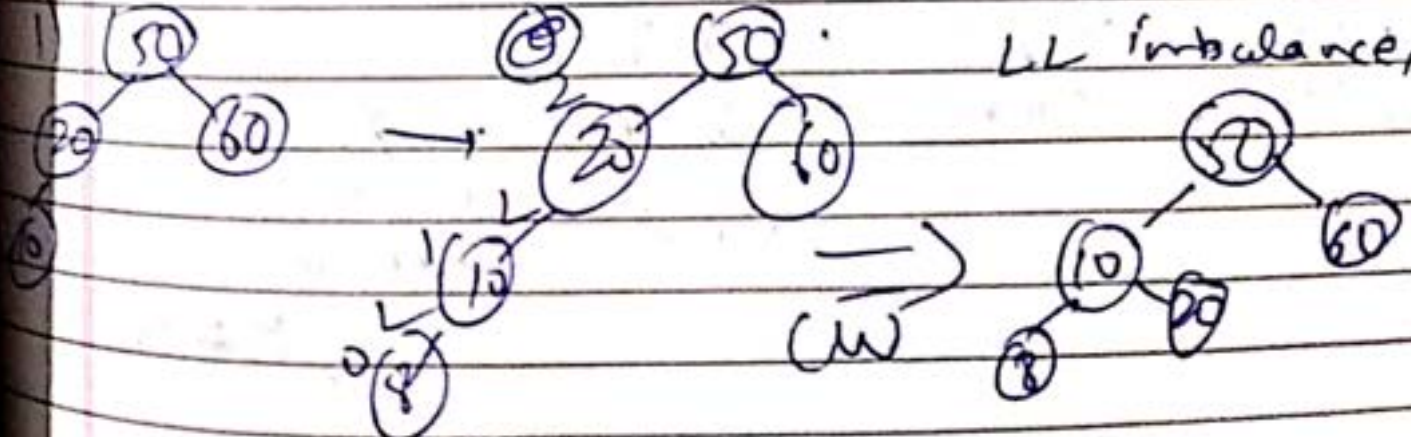
RL imbalance (2 rotation)



LR imbalance (2 rotation)

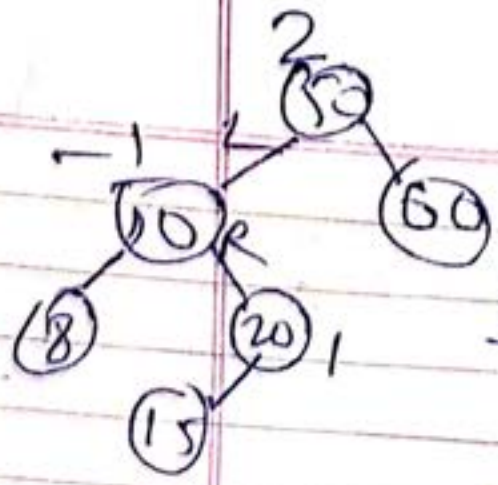


Insert in AVL 50, 20, 60, 10, 8, 15, 32, 46, 11, 48

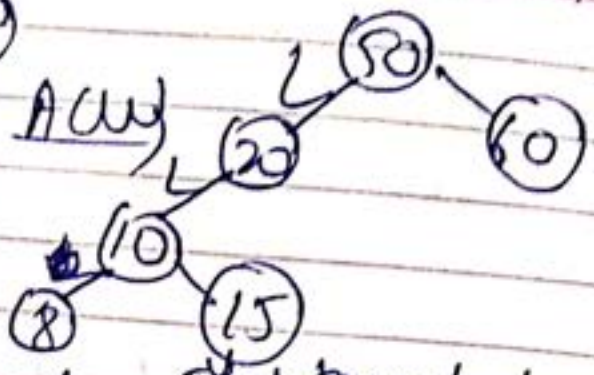


Scanned with CamScanner

LL → CW
RR → ACW
LR → ACW, CW
RL → CW, ACW



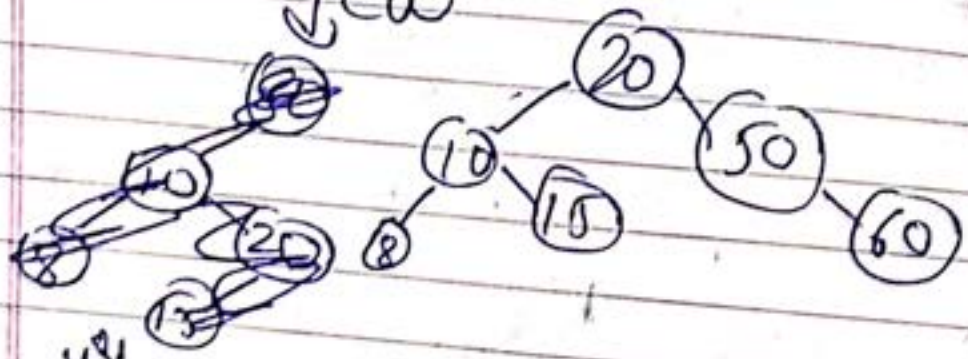
ACW,



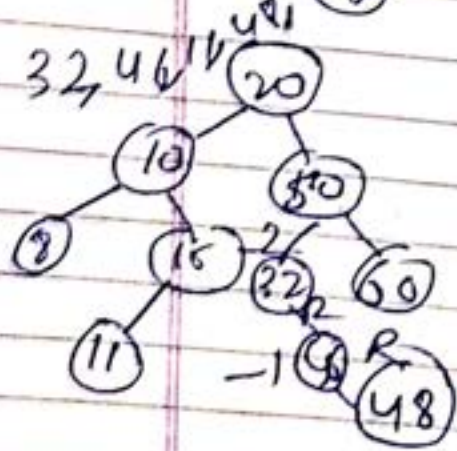
So

Improvement.

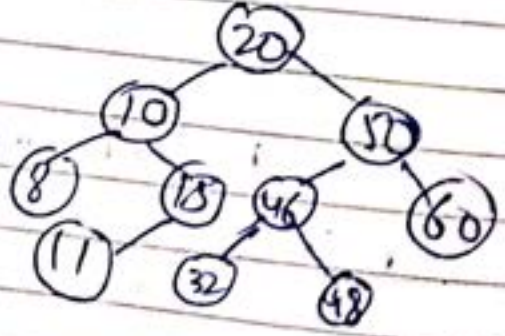
CW



32 46



ACW,

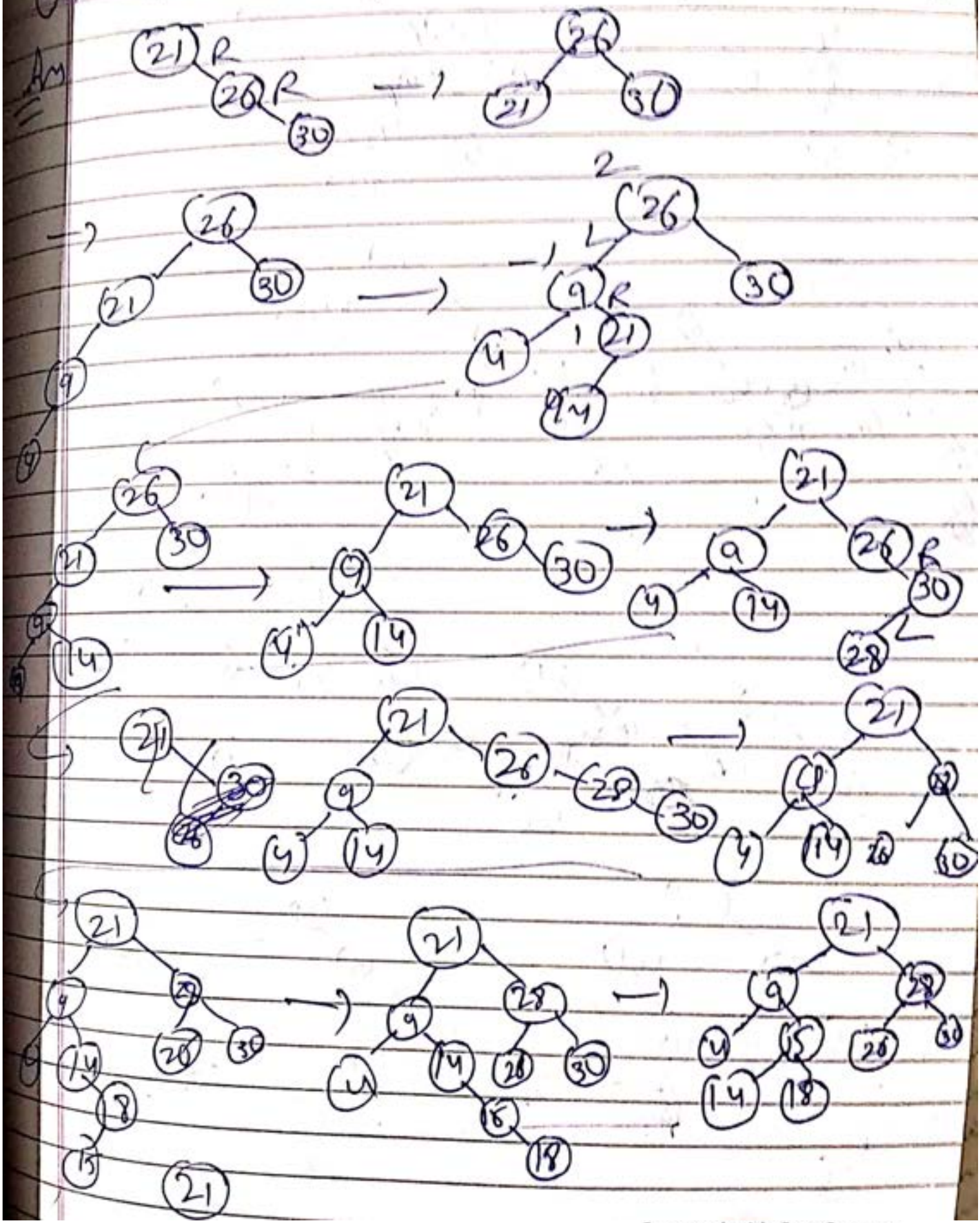


	BST ~~Unbalanced~~	Balanced or AVL
Search	O(n)	BBST O(log n)
height	O(n)	O(log n)
Insertion	O(n)	O(log n) → c g(log n) + c to insert to Balance rotations

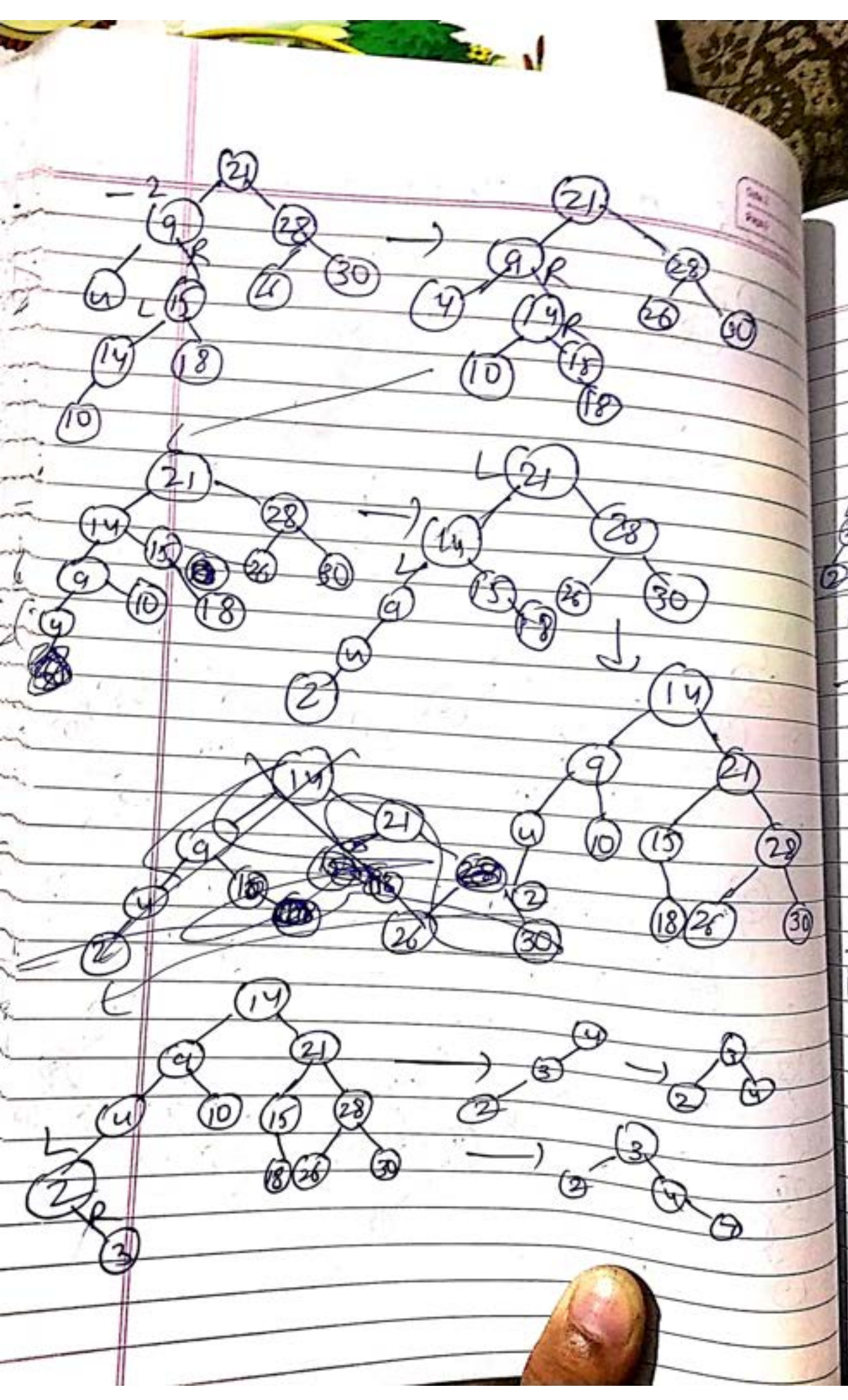
Scanned with CamScanner

Q. 21, 26, 30, 9, 4, 14, 28, 18, 15, 10, 2, 3, 7

Ans



Scanned with CamScanner



Max & m

→ Max

Min → If

N(H)

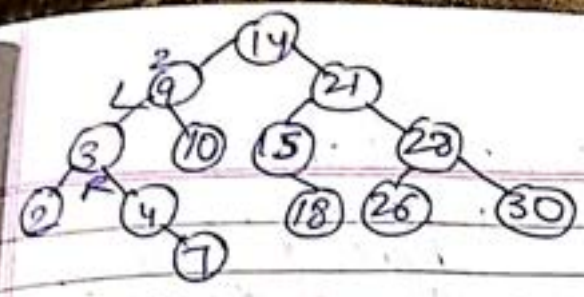
N(H) =

N(1)

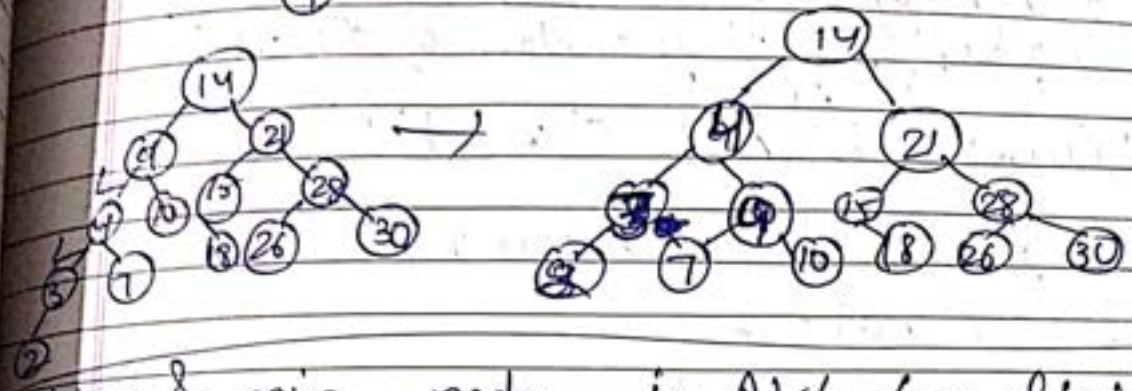
N(2)

N(3)

Scanned with CamScanner



Date / Page 173



Max & min nodes in AVL tree of height n

Height max

→ Max $0 \rightarrow 1$

$1 \rightarrow 1+2^1=3$

$2 \rightarrow 1+2^1+2^2=7$

$3 \rightarrow 1+2^1+2^2+2^3=15$

$h - 1+2^1+2^2+2^3+ \dots +2^h$

$\frac{2^{h+1}-1}{2-1} = 2^{h+1}-1.$

Min → If one side h then other $\geq (h-1)$

$N(H)$ = min no nodes of height h

$N(H) = N(H-1) + 1 + N(H-2)$

$N(1) = 2, N(0) = 1$

$N(2) = N(1) + 1 + N(0)$

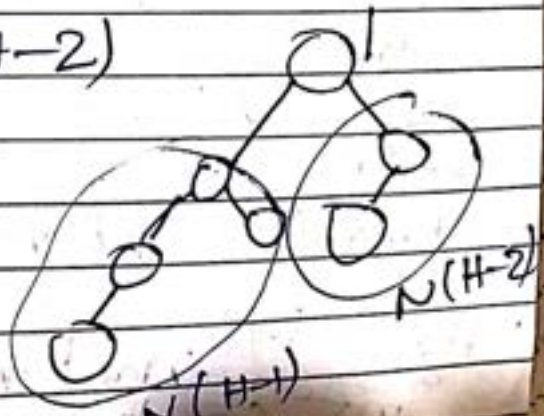
$= 2 + 1 + 1$

$= 4$

$N(3) = N(2) + 1 + N(1)$

$= 4 + 1 + 2$

$= 7$



Scanned with CamScanner

$N(4) = N(3) + 1 + N(2) = 7 + 4 - 2$

Q. A binary tree 'T' has 'n' leaf nodes. The # of nodes of degree (2) in 'T' is:

a) $\log_2 n$ b) $n-1$ c) n d) 2^n

tree → 1 node of degree 2

leaf = 2

2 nodes of degree 2

leaf = 3

3 nodes of degree 3

leaf = 4 So b)

Q. In a binary tree, # of internal nodes of degree '1' is 5, & # of internal nodes of degree 2 is '10'. The # of leaf nodes are

a) 10 b) 11 c) 12 d) 15

degree 2 degree 2 = 3

2

General $N(n) = N(n-1) + 2 - 1$

Degree '1' won't make any difference

Just check by examples also

Scanned with CamScanner

Gate IT 2008

Q) Which of the following is true about search times

	AVL	BST
a)	$\Theta(\log n)$	$O(n)$
b)	$\Theta(\log n)$	$\Theta(n \log n)$
c)	$\Theta(n)$	$\Theta(\log n)$
d)	$\Theta(n \log n)$	$O(n)$

↑ worst

a) True — Θ means = and implies that height always $\log n$ whether best or worst case

b c d false

Gate 1998

Q) Which of the following is false

a) A tree with 'n' nodes has (n-1) edges

b) A labeled rooted binary tree can be uniquely constructed given its post order & preorder traversal

c) A complete binary tree with 'n' internal nodes has (n+1) leaves

d) The maximum number of nodes in a binary tree of height 'h' is $2^{h+1} - 1$

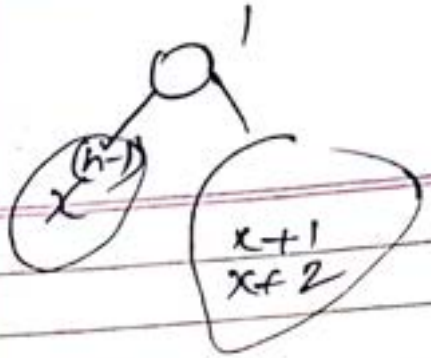
Ans (b)

Gate 2005

Q) In a binary tree, for every node the difference b/w the number of nodes in the left & right subtrees is at most 2. If the height of the tree is $h > 0$, then the minimum number of nodes in the tree is.

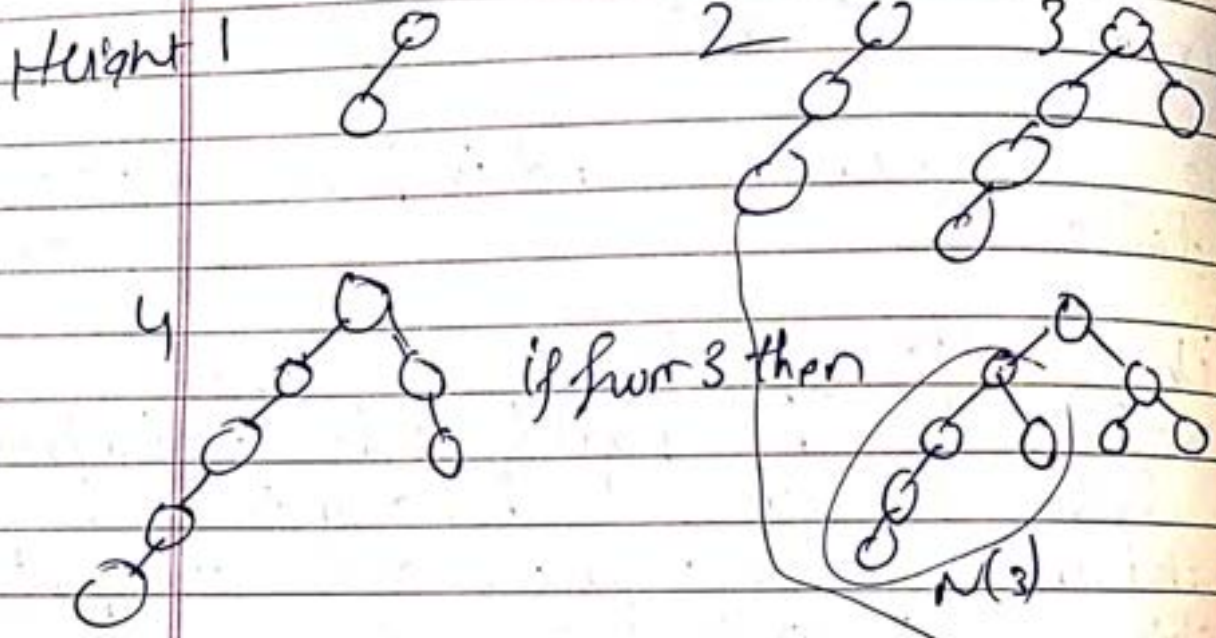
a) 2^{h-1} b) $2^{h-1} + 1$ c) $2^h - 1$ d) 2^h.

Scanned with CamScanner



So $N(h) = N(h-1) + 1 + (N(h-1) - 2)$

$N(h) = 2N(h-1) - 1$



$N(2) = 2N(1) - 1 = 2(2) - 1 = 3$

$N(3) = 2(N(2) - 1$

$= 2 \times 3 - 1$

$= 5$

From N(2) & N(3)

$Ans = b) \; 2^{n-1} + 1$

Scanned with CamScanner

AVL Tree $N(h) = N(h-1) + 1 + N(h-2)$

~~$N(h) = 2$~~

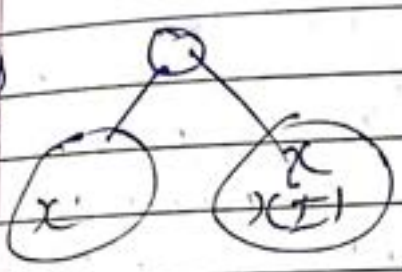
Date / Page 177

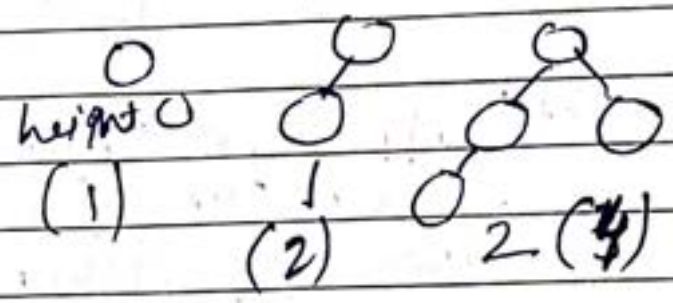
Gate 1997

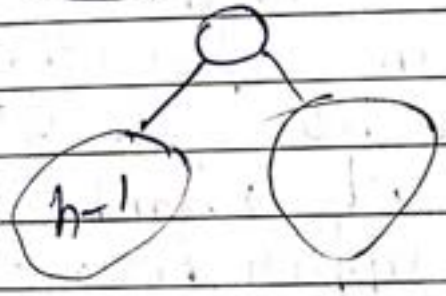
Q) A size balanced binary tree is a binary tree in which for every node, the difference b/w the # of nodes in the left & right subtree is atmost 1. The distance of a node from the root is length of the path from the root to the node. The height of a binary tree is the maximum distance of a leaf node from the root.

A) Prove by induction on h, that a size balanced binary tree of height 'h' contains at least 2^h nodes.

Ans







$N(h) = N(h-1) + 1 + (N(h-1) - 1)$

$N(h) = 2N(h-1)$ — (1)

$N(h-1) = 2N(h-2)$ — (2)

$N(h-2) = 2N(h-3)$ — (3)

So $N(h) = 2(2N(h-2)) = 2^2 N(h-2) = 2^3 N(h-3)$

$N(h) = 2^K N(h-K)$

$N(1) = 2N(0) = 2$, So $K = h-1$:

$N(h) = 2^{h-1} N(h-h-1) = 2^{h-1} \times 2$

$N(h) = 2^h$

Scanned with CamScanner

Q Max height with such a tree with 4 nodes.

Ans $h \to 2^h$

So $h = 2 \therefore 2^2 = 4$

Also let's do A) by induction

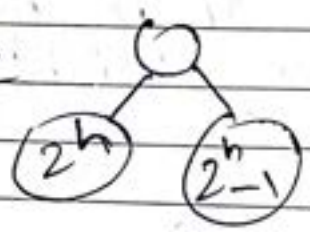
So if $N(h) = 2^h$ (given) Statement

then $N(h+1) = 2^{h+1}$ (To P)

$N(h+1) = 2^h + 2^h - 1$

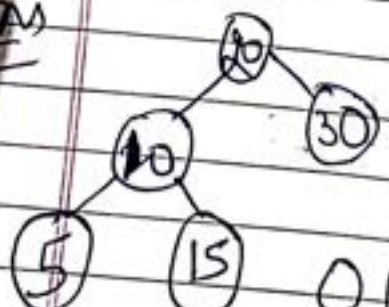
$= 2 \cdot 2^h$

$= 2^{h+1}$



Gate 2014 Q Suppose we have a balanced binary search tree 'T' holding 'n' numbers. We are given 2 numbers 'L' & 'H' & wish to sum up all the numbers in 'T' that lie b/w 'L' & 'H'. Suppose there are 'm' such numbers in T. If the tightest upper bound on time to compute the sum is $O(n^a \log^b n + m^c \log^d n)$, the value of a+10b+100c+1000d is ?

Ans



Let's say L = 4

H = 16

So sum = 5+10+15 = 30

m = 3

$O(5^a \log^b + 3^c \log^d 5)$

Now

Take

m n

So

Ans 10

Other

Experi

a+b

Find

the

So

Pre

IN

Post

= a

→

Now b/w 4 & 16

Take { 5, 6, 7, ..., 15 } & search in tree O(log n)

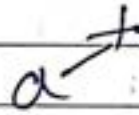
m numbers so m log n

So a=b=0, c=d=1 → basically (H-L) log n

Ans 1000 + 100 = 1100 (so this query is saving time)

Other way, sort inorder O(n)
then binary search O(n) + O(log n)
= O(n)
but this is not relevant to this ques

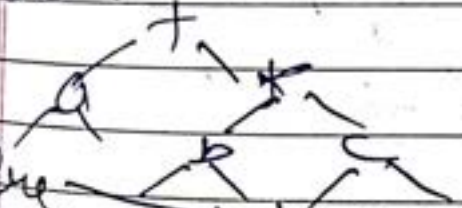
Expression trees → a+b



Pre → + ab
IN → a+b
Post → ab+

a+b*c
Find b*c (high precedence)
then add

So

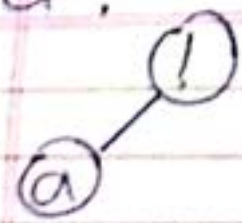


Pre → + a*bc → Prefix expression
IN → a+b*c → Infix
Post → abc*+ → Post

a → ⊖ (symb) In → -a

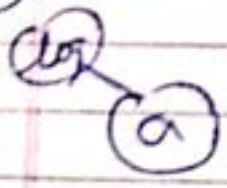
if ⊖ then a-X

a!

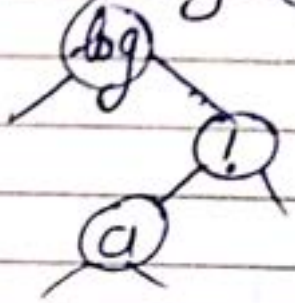


Inorder - a!

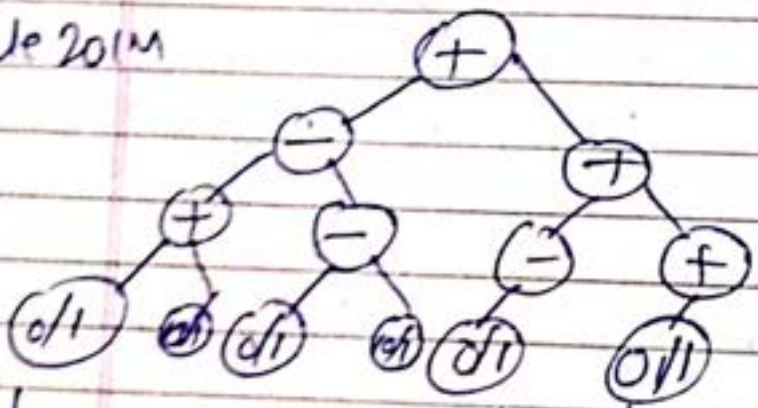
log a



log (a!)



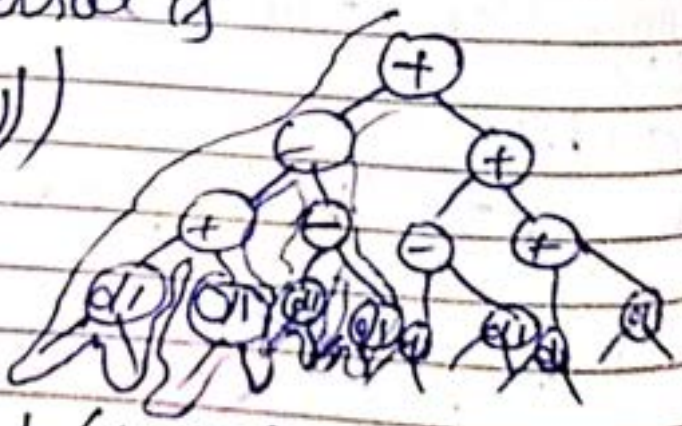
Gate 2014



when max value (+ve) of 1 of last level

Ans Inorder traversal is

$$((0!)+(0!)-(0!+(0!))$$
$$+((0!)-(0!)+(0!+0!))$$

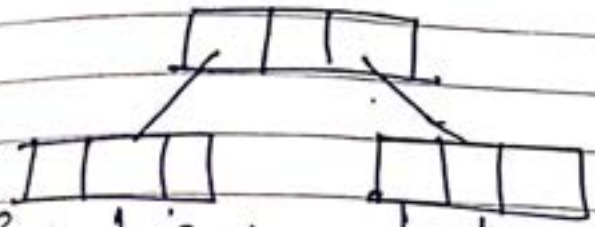


So max = $(2-(-1))+(1+2)$

$= 6$

Scanned with CamScanner

→ Various tree representations



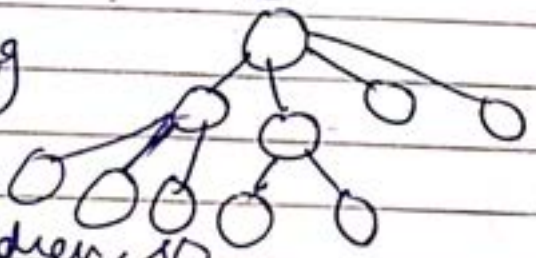
OK for binary but not for n-ary (Let's say n=10)

So LCRS representation is also there.

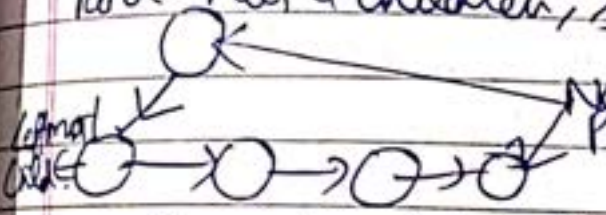
L – Leftmost C-child

R – Right S – Sibling

Let's say for eg

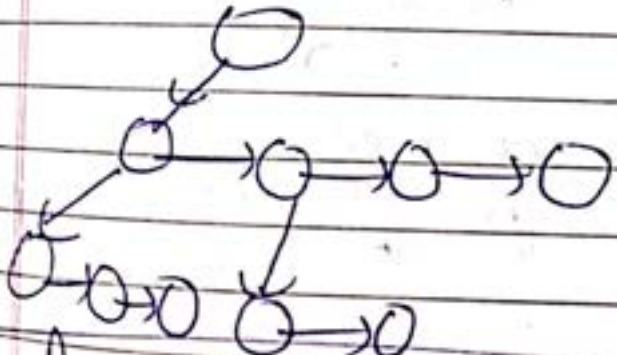


Root has 4 children, so

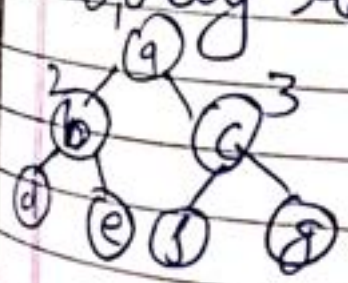


And leftmost child has pointer to next right child & so on

So complete tree



Array representation



indexes

a – 1

LC = b – 2i C → 2i+3 = RC

Multiplying by 2 simply means left shifting in binary

by 2i for i=2 is 4 & 2i+1=5

10 100 101

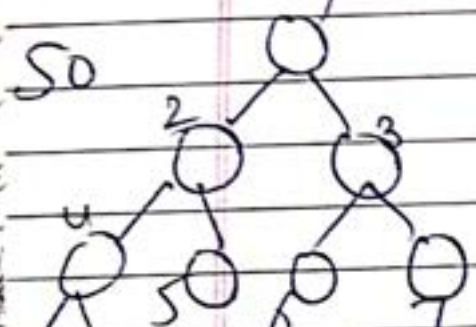
Scanned with CamScanner

Parent = $\frac{i}{2}$ → no of elements
→ right shift

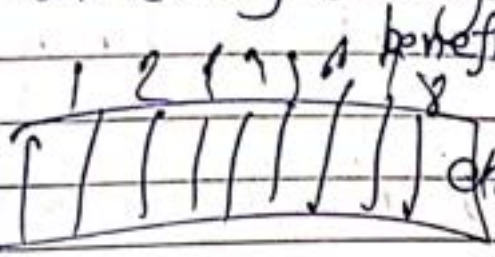
$\frac{100}{2} = 10$ i.e 100 right shift
10/1 → 10

1	2	3	4	5	6	7
a	b	c	d	e	f	g

Representing a heap as array is very beneficial



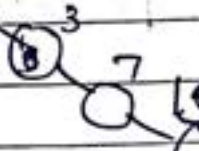
So



dk.

and heap will always grow in array from left, so array space utilized

But, if tree is like



it's not beneficial (15 elements for 4 values in array)

ex parent of 2^{n-1}

$O(2^n)$

So not advisable

Advisable only when tree is going to grow in level order

Scanned with CamScanner

Nested form

(b a c) (a b c)

in root L R form

a(bde)(cfg)

root, left subtree, right subtree

Types to form
1) Normal (pointers, structures)
2) LCRS (n-ary n>>2)
3) Array (heaps)
4) Nested

Gate 2006
Q (XYZ), Y and Z can be NULL, which of the following is valid binary tree
a) (1 2 (4 5 6 7)) b) (1 ((2 3 4) 5 6) 7)
c) (1 (2 3 4) (5 6 7)) d) (1 (2 3 NULL) (4 5))

Array eg

(1 2 NULL) (1 NULL NULL)

(1 (2 4 NULL) (3 NULL 5))

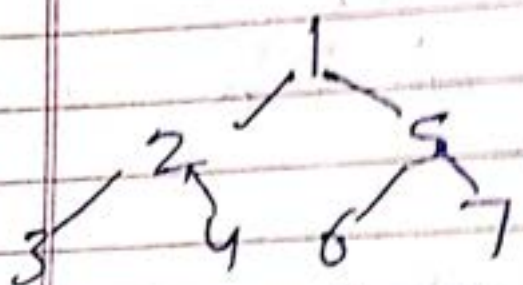
a) 1 / 2 — Not possible ✗

b) 1 → 2 → where to place 5, 6 ✗

root(1) select → No meaning ✗

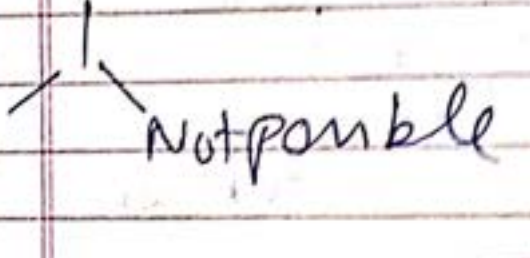
Scanned with CamScanner

c) (1(234)(567))



Valid

d) (1(23 NULL)(45))

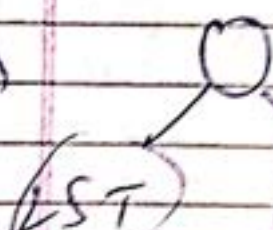


Not possible

Gate 2005 Q-1 The numbers 1, 2, ..n are inserted into a BST in some order. In resulting tree, the right subtree of root contains 'p' nodes. The first number to be inserted in the tree must be

(a) p (b) p+1 (c) (n-p) (d) (n-p+1)

Ans



1, 2, ... p ... n

Out of n are at right of n-p-1 at left and 1 at root

So root must be (n-p-1+1) = n-p

So (1, 2, ... n-p-1)(n-p)(n-p+1 ... n)

is true.

PS- Same for AVL also

Scanned with CamScanner

⌈ ⌉ ceil
⌊ ⌋ floor

Gate-IT-2006

Date / Page 185

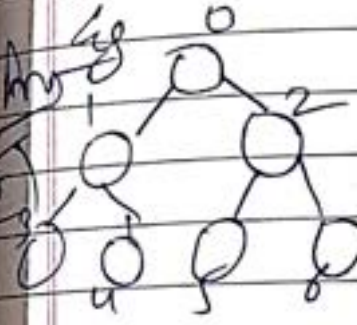
Q. An array 'x' of 'n' distinct integers is interpreted as a complete binary tree. The index of the first element of array is 0. The index of parent of element x[i], i≠0 is

a) $\lfloor \frac{i}{2} \rfloor$ b) $\lceil \frac{(i-2)}{2} \rceil$ c) $\lceil i/2 \rceil$ d) $\lceil i/2 \rceil - 1$

Q. If the root node is at level 0, the level of element x[i], i≠0, is

a) $\lfloor \log_2 i \rfloor$ b) $\lceil \log_2 (i+1) \rceil$ c) $\lfloor \log_2 (i+1) \rfloor$ d) $\lceil \log_2 i \rceil$

Ans



Let's take 4

$\frac{i-2}{2} = \frac{1}{2}$ $\lfloor \frac{i}{2} \rfloor - 1 = 1$

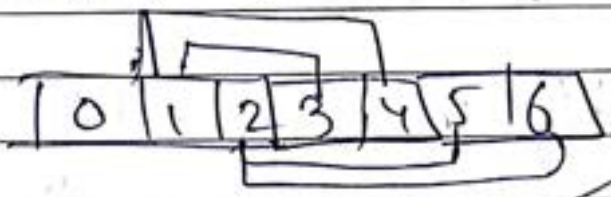
a, c removed from options

Now let's take 3 $\frac{i-2}{2} = 0.5$ $\lceil 1.5 \rceil = 2$

& $\lceil \frac{i}{2} \rceil - 1 = 2 - 1 = 1$ (so b, d both are actually same

$\lceil \frac{i-2}{2} \rceil = \lceil \frac{i}{2} \rceil - 1$

General



$i-x-1$

∴ Parent & left child $3-1-1=1$ element

" " right " $4-1-1=2$ element

if parent in 2nd layer

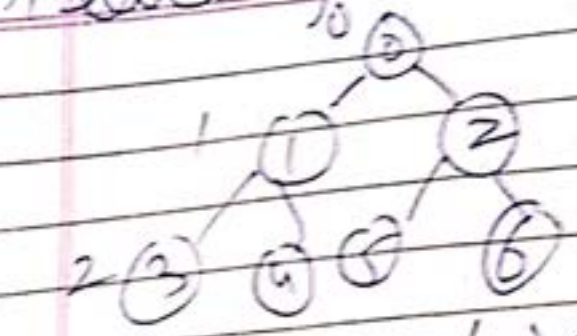
illy device for 3rd & then nth layer

$x \le i-x-1 \le x+1$

In left child $x = \frac{i-1}{2}$ Right $x = \frac{i-2}{2}$

Scanned with CamScanner

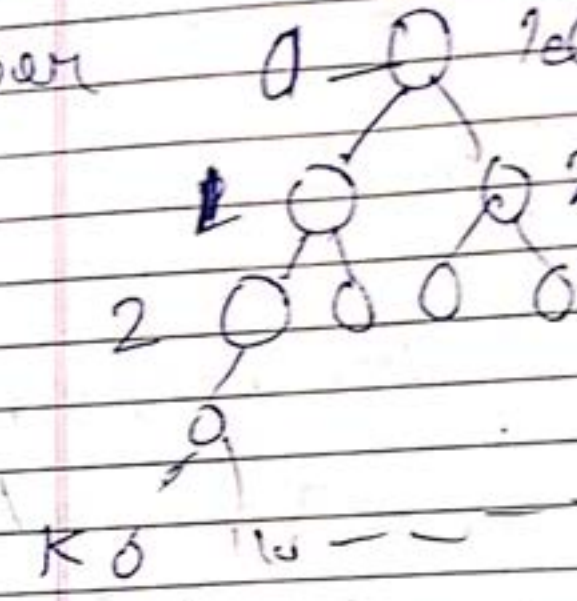
(b) level of x[i]



for i=4

for i=5

Proper



2^{k+1} = k level if k is the ans

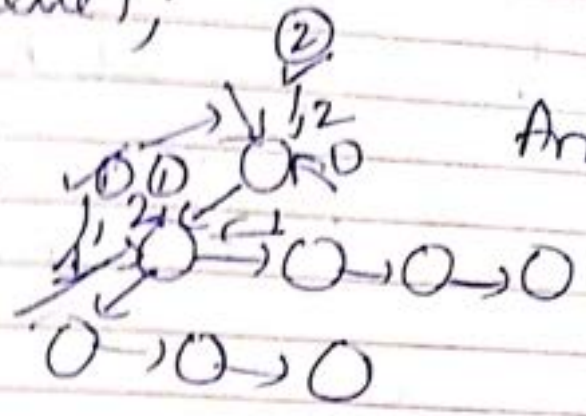
Q2 2014

left most - right sibling representation is used
Each node of tree is of type treeNode

```
typedef struct treeNode* treePtr;
struct treeNode {
treePtr leftMostChild, rightSibling;
};
int DoSomething(treePtr tree) {
int value = 0;
```

```
if (tree != NULL) {
    if ((tree->leftMostChild == NULL)
        value = 1
    else {
        value = DoSomething(tree->leftmostChild);
        value = value + DoSomething(tree->rightSibling);
    }
    return(value);
}
```

Let's take eg

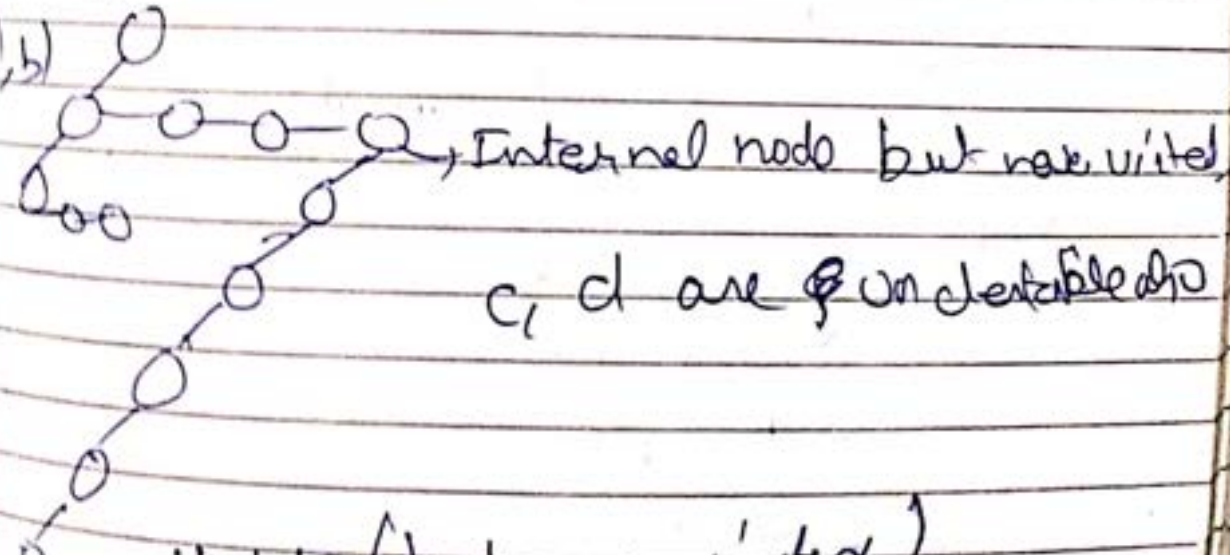


options

a) function will call fn of internal nodes
b) " " return height of tree
c) and fn of right sibling node
d) fn of leaf nodes counted

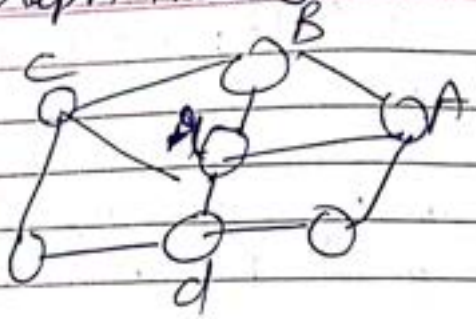
All false, How?

a), b)



Scanned with CamScanner

Graphs (Graph Theory) →



Eg WhoW, Social NW site

Search → A particular way
Traversal → Passing from all vertices

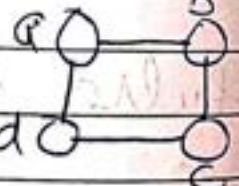
Let's say for a Breadth first search at level 1 is a, b, c, d

Ily BFS – 2 etc.

DFS – go as deep as possible then ——— etc.

Representation of Graphs (2 ways)

1) Adjacency matrix



	a	b	c	d
a	0	1	0	1
b	1	0	1	0
c	0	1	0	1
d	1	0	1	0

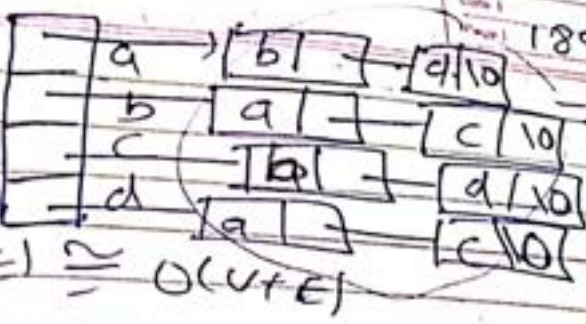
weights if weighted graph

$O(n^2)$

$O(V^2)$

(adjacency list)

2) Linked list



Array of pointers (size = # of vertices)

$O(V+2E) \cong O(V+E)$

If dense graph, a lot of edges $E = O(V^2)$ use (1)

If sparse $E = O(V)$ (2)

Introduction to BFS & DFS



BFS 2 words Visited → seen vertex
explored → seen + adjacent vertex

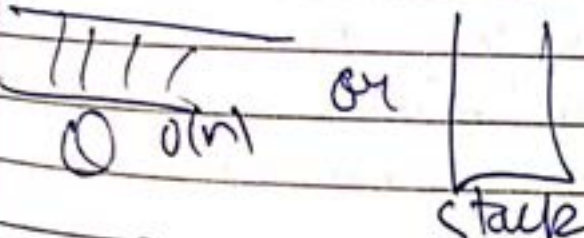
	Visited	Explored
BFS	0	0 ~
	1	0 ~
	1	1
DFS	0	1 X (Not possible)

Visited

1	2	3	4	5	6	7	8
1	0	1	0	0	0	0	0

Initially Array

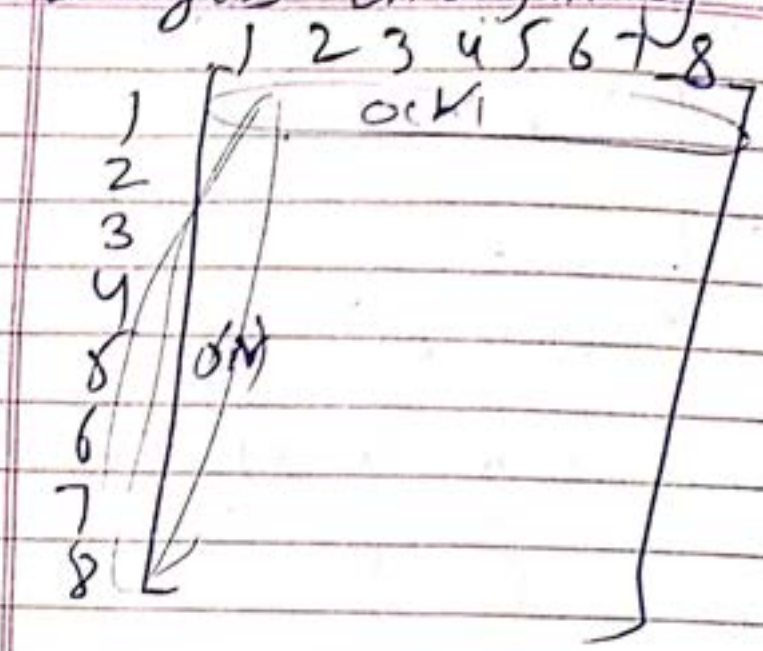
explored



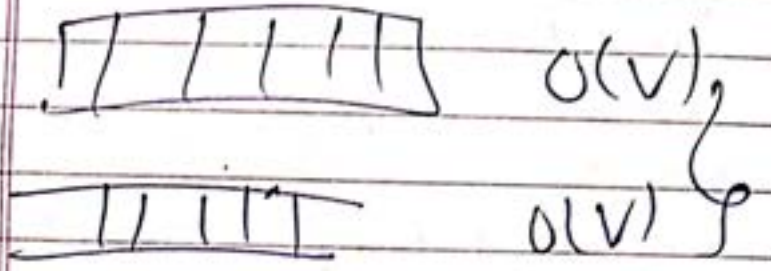
Now when queue used BFS O(n)
Stack " " DFS O(n)

Scanned with CamScanner

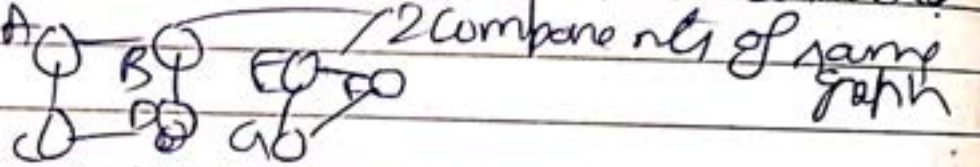
BFS analysis on adjacency matrix implementation



Space



A graph can be connected / not-connected



How to know if connected or not
Traverse the graph (let's say starting from A) then if G will be not visited in adjacency visited matrix array.
So we come to know we not connected.

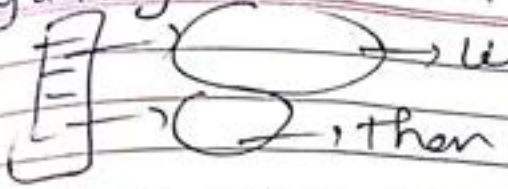
```
BFS (G, n) {
for i = 1 to n do
    visited[i] = 0;
for i = 1 to n do
    if (visited[i] = 0) then BFS(i);
}
```

BFS on E, F, G

Scanned with CamScanner

In both DFS & BFS Time complexity same
BFS & DFS can be applied to directed graph as well

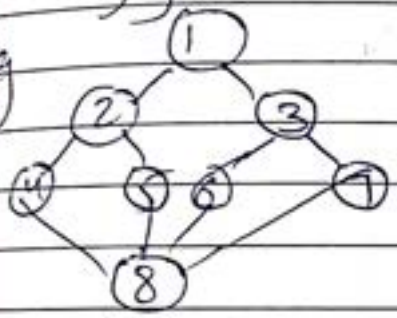
adjacency list

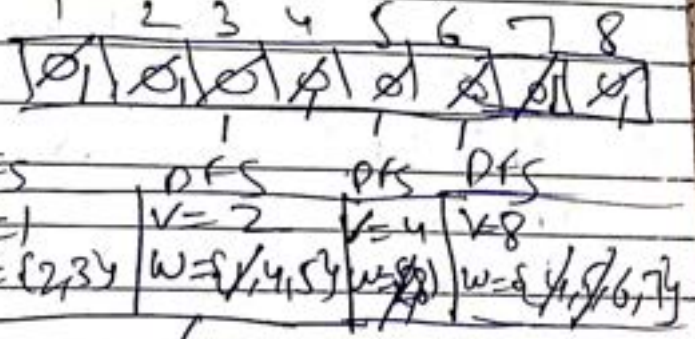


Note - Each vertice is visited only once, so time complexity O(E+V)
traversing for vertices

In adjacency matrix V^2

```
DFS(V){
visited[w]=1;
for each vertex w adj to v do{
    if(visited[w]==0] then
        DFS(w);
}}
```





then whole popped out(return)

Space Complexity — O(V) (Stack)

in adjacency list ←

Time Comp → O(E+V)

worst case



adjacency matrix $O(V^2)$

Scanned with CamScanner

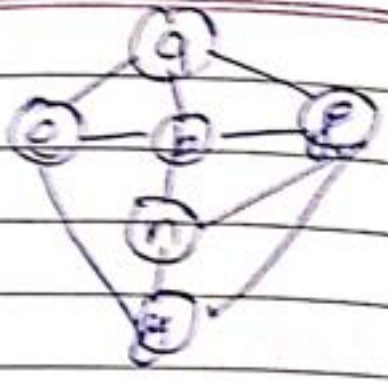
DFS possible sequence if DFS started

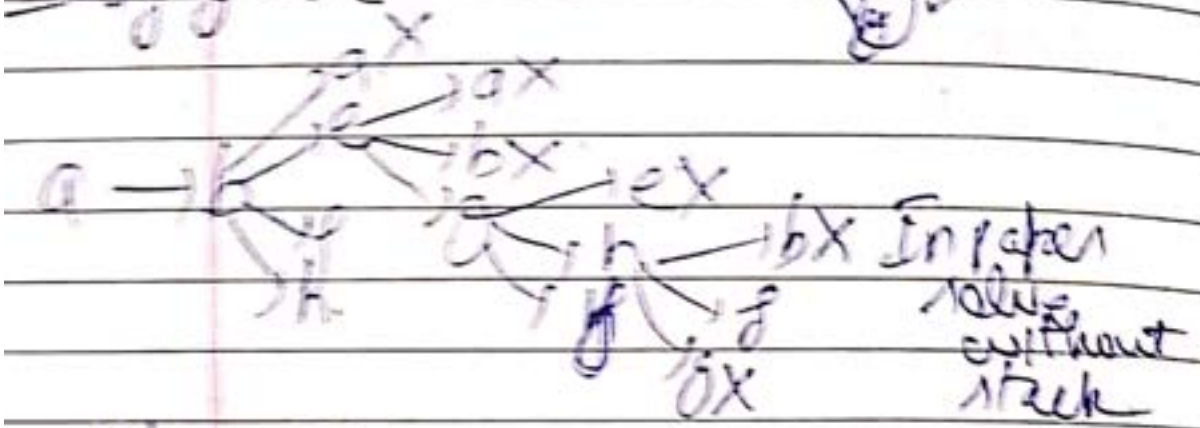
I a b e g h f

II a b f e g h

III a b f h g e

IV a f g h b e





In path value without stack

abegfh ✓

abfeX

abfhge ✓

afghge ✓

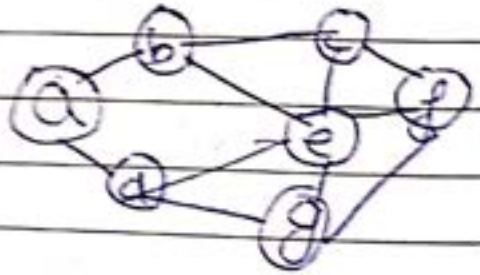
Path which is not possible DFS

1) abefdgc X

2) abefcgd ✓

3) abgefdc ✓

4) adbcefg



2) a b e f c g d

we comeback to f to go to g (: f can't go anywhere else)

(can't go anywhere else)

4) a d b c

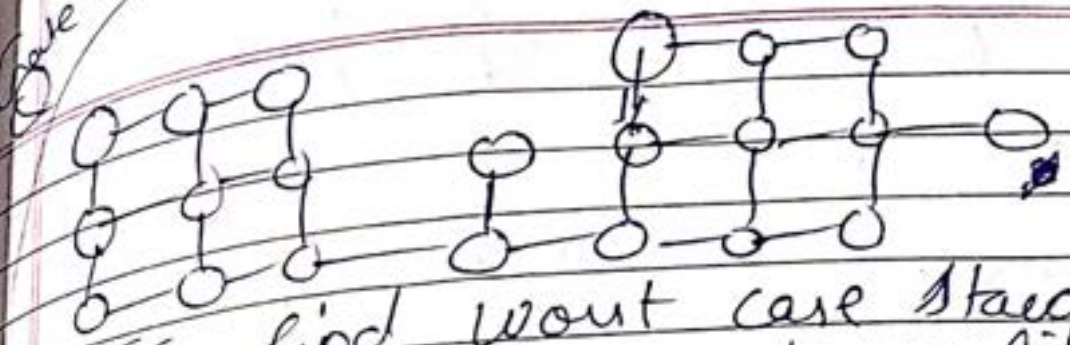
comeback to a not possible : d has e f to go

Scanned with CamScanner

Alter → (find longest path)

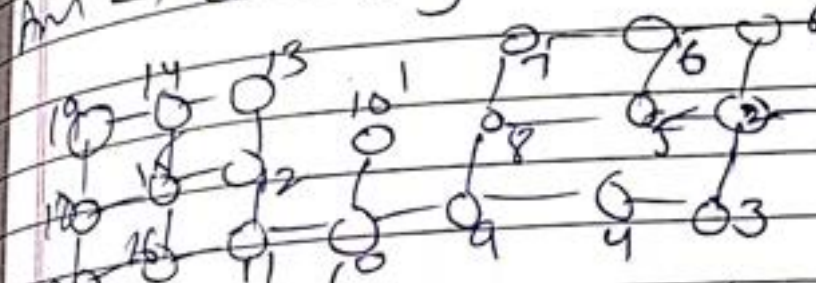
Date / Page 193

Ques:



DFS, find worst case stack condition. Start from any node you like.

Ans - Let's say start from A(1)



P.S You can goto 6' & 10' but that won't affect stack size because that will eventually return and memory freed.

Ans (19)

Gate 2006

Q- Consider a DFS of an undirected graph with 3 vertices P, Q and R. Let discovery time d(u) represent the time instant when the vertex 'u' is first visited, and finish time f(u) represent time instance when vertex u is last visited. Given that

d(P) = 5 units f(P) = 12 units

d(Q) = 6 units f(Q) = 10 "

d(R) = 14 11 f(R) = 18

What is true about graph?

Scanned with CamScanner

5 6 10 12 14 16
d(P) d(Q) f(Q) f(P) d(R) f(R)

SU (P) → (Q)

(R)

So 2 component graph

Actually this DFS is BFT ≥

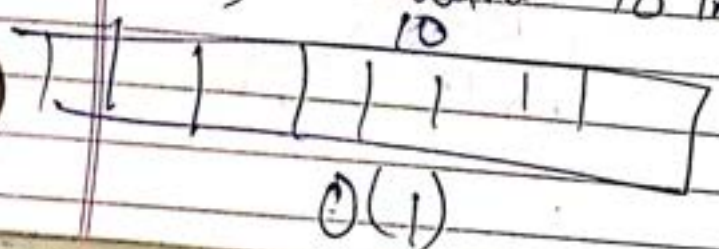
DS → Major operations
Insertion
Searching ✓ (Most used if static dataset)
Delete

Searching 1) Array (unsorted) – O(n)
2) " (sorted) – O(log n)
3) LL – O(n)
4) BT – O(n)
5) BST – O(n)
6) AVL → O(log n)
7) PriorityQueue (min & max) – O(n)

So to make searching more optimum hashing is introduced O(1) on average

Before hashing people used DAT (Direct Address Table)
similar to array

Let's say we want to insert 100 keys



O(1)

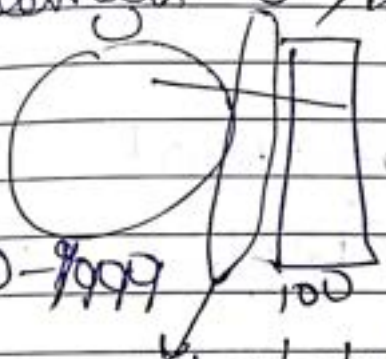
So if we want to insert data at 10th key
so searching with key given O(1)

But
→ keys
→ direct
size
Disadv
→ if keys
data
1
→ size of
So h
Hashi
Let's
of ea
O
Searc
g →
hash
I
So
for

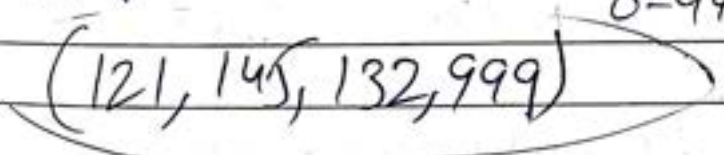
Scanned with CamScanner

But DAT is efficient if
→ keys are not large (no number (len of key))
→ direct proportionality b/w # of keys & size of array
Disadv
→ keys are available in large # and data is less, then also waste of space
→ size of array is as big as # of keys
So hashing used instead

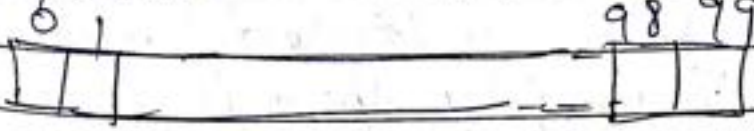
Hashing →
Let's say we want to store universal set of elements U, but at some time only few are accessed



Search time O(1) (we just say element is present or not)
Eg → Suppose we insert



hash function h(0-9999) → (0-99)



So mod(100) here.
for eg if 121 comes 21th place
145 45th place like 132, 999

So insertion - O(1)
Now searching

So if we want to find 150
10 150%100 = 50 = NULL
So O(1)
Ily 45, 145%100 = 45 = O(1)

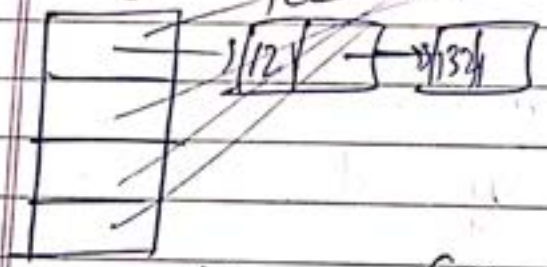
Problem → collision

collision → 121, 1321 goes to same place, then collision happens

Various techniques to solve problem

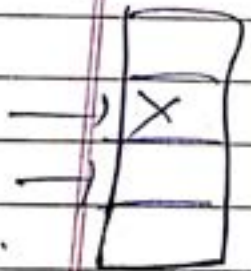
(i) Better hash function
(but total collisions never be 0)

(ii) Chaining (Good)



(Better for I, D, S)

(iii) Open addressing (Better for I, S)
No deletion



If collision when we want to place all, then again hashing performed and the empty space is found out.

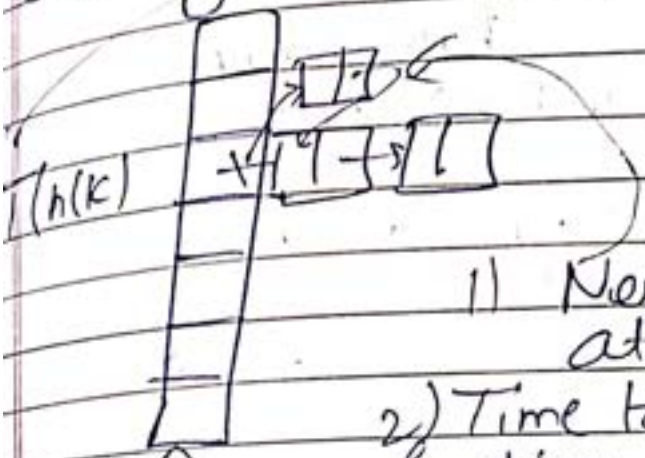
a) Linear probing
b) quad "
c) Double hashing

Scanned with CamScanner

Date / Page 97

Main diff b/w
1) Chaining needs extra space but collision won't happen
2) Open addressing needs no extra space but collision may happen

Chaining



→ Time = Time taken to compute the hash function + Time taken to insert = constant time.

Because
1) New insertion take place at beginning (constant time)
2) Time taken to compute hash function is just mathematical formula, so constant time.

I → O(1) search
Worst case = all n places filled so searching needs O(n) time, but from defn of chaining we know this n is << actual size of universal set U

Worst case D → O(n)

Average search $\theta(1+\frac{n}{m})$

$\alpha = \frac{n}{m}$ → load factor

$\theta(1+\alpha) \simeq \theta(1)$

n = elements present in hash table
m = size of hash table

In open addressing, after deletion rearranging required

Average deletion time
$\Theta(1+\alpha) \cong \Theta(1)$

Disadv ⊗ → Elements are not stored inside table, but in outside LLs; so extra space required

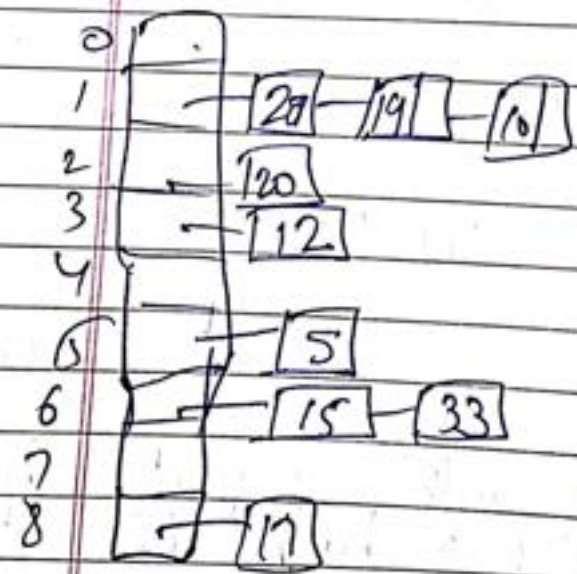
Gate 1996

Q. An advantage of chained hash table over open addressing scheme is
a) Worst case complexity of search is less
b) Space used is less
c) Deletion is easier
d) none

2014.

Q. $h(k) = k \bmod 9$, hash table has 9 slots, chaining is used
Keys: 5, 28, 19, 15, 20, 33, 12, 17 & 10. Then max, min & average chain length in hash table.

So



max – 3
min – 0
avg – (i.e α)
$\alpha = \frac{9}{9} = 1$

Another way of avg
$\frac{0+3+1+1+0+1+0+1+2}{9}$
$= 1$

Gate 1997

Q. Consider
where
colli
such
key
bucke
is in
Valie
a) what
num
b) What
has
c) what
occu

Ans a) Sho

So pro

and

$\left(\frac{n}{n}\right)$

b) for K=

for

$\left(\frac{n}{n}\right)$

c) $\left(\frac{n}{n}\right)\left($

Scanned with CamScanner

Date 1
Page 199

Gate 1997

Q. Consider a hash table with 'n' buckets, where chaining is used to resolve collisions. The hash function is such that the probability that a key value is hashed to a particular bucket is $(1/n)$. The hash table is initially empty & 'k' distinct values are inserted in the table.

a) What is the probability that bucket number '1' is empty after k insertions?

b) What is probability that no collision has occured in any of the operations?

c) What is the probability that first collision occurs at kth insertion?

Ans a) Prob $= \frac{1}{n}$ dimaging so each bucket equally likely

so prob of not inserting in each $= 1 - \frac{1}{n} = \frac{n-1}{n}$

and total k insertions so

$$\left(\frac{n-1}{n}\right) \cdots \left(\frac{n-1}{n}\right)\left(\frac{n-1}{n}\right) = \left(\frac{n-1}{n}\right)^k$$

b) for $k=1$, no chance of collision so $\frac{n}{n} = 1$

for 2nd case $\frac{n}{n}\left(\frac{n-1}{n}\right)$ and so on, so

$$\left(\frac{n}{n}\right)\left(\frac{n-1}{n}\right)\left(\frac{n-2}{n}\right) \cdots \left(\frac{n-k-1}{n}\right)$$

c) $\left(\frac{n}{n}\right)\left(\frac{n-1}{n}\right)\left(\frac{n-2}{n}\right) \cdots \left(\frac{n-(k-2)}{2}\right)\left(\frac{k-1}{n}\right)$

$\therefore$ k-1 are filled & it has to collide

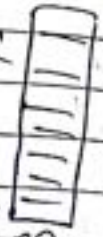
Scanned with CamScanner

24/9/20

Open Addressing (or Closed Hashing)

- Don't save elements outside but inside table itself
- At any point we can't insert more than m

$0 \leq \alpha = \frac{n}{m} \leq 1$ in open addressing



In chaining $0 \leq \alpha \leq \infty$

- We know universal set U has lot more keys than m
- Probing → find location where to insert element in hash table
- If collision first time inserting we apply second time probing with a slight modification in hash function.

$h: U \to \{0, 1, \ldots, m-1\}$

$h: (U \times \{0, 1, 2, \ldots, m-1\}) \to \{0, 1, \ldots, m\}$

(second probing, first probing)

- First time we check with 0 if collision second time with 1 and so on
- This sequence of proceeding is called probe sequence.

Would hine

- If collis

$h(k_1) = 2$
$h(k_1, 1) =$
$h(k_1, 2) =$

Let's say tab

$h(k_1, 0) =$
$h(k_1, 1) =$
$h(k_1, 2) =$
$h(k_1, 3) =$
$h(k_1, 4) =$
$h(k_1, 5) =$
$h(k_1, 6) =$
$h(k_1, 7) =$

- Deletion

Let's say
& we se
Semph
Now sea
Seque
$h(K, 0) = 2$
We stop at
1) K=5 or
2) empty c
3) End of

- Worst time to search $O(m) = 20$
- It of collisions are also counted by hash function

$h(k_1) = 2$ → 1 collision
$h(k_1, 1) = 6$ → 1 collision → index 6
$h(k_1, 2) = 3$ → Success

let's say table full, then

$h(k_1, 0) =$	2 → probe sequence
$h(k_1, 1) =$	6
$h(k_1, 2) =$	3
$h(k_1, 3) =$	4
$h(k_1, 4) =$	5
$h(k_1, 5) =$	1
$h(k_1, 6) =$	7
$h(k_1, 7) =$	0

- Deletion will degrade search efficiency

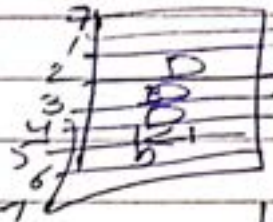
let's say element k_1 to inserted & probe sequence is 2, 6, 3, 4 full
Empty so inserted at 5
Now searched using probe sequence
$h(K, 0) = 2$, ... $h(K, 4) = 5$
We stop at
1) K = 5 or
2) empty cell or
3) End of table

0	
1	
2	a
3	e
4	d
5	K
6	b
7	

Now, let's say someone deletes C (key 3), now how search k_1 again, then
we search for k_1 again, then it stop at 3 so problem solved

Sol:- Whenever we delete any element we will put something else not NULL, let us say we put 'D', so we can identify deleted elements in this manner.

worst case 2, 6, 3, 4 all are deleted



So now when we search for K, we need 5 jumps but ideally it should be 1 jump. So in this manner it affects

3 techniques of open addressing

1) Linear probing
2) Quadratic ,,
3) Double Hashing

1) linear probing

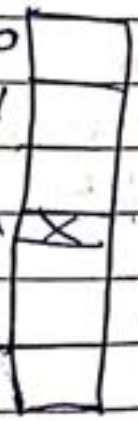
$h: U \rightarrow \{0 - - - m-1\}$

$h(K) = a$ occupied

So $h'(K, i) = (h(K) + i) \bmod m$

Slight modification in hash function

So $h'(K, 1) = (a+1) \bmod m$



linear because $a \rightarrow (a+1) \rightarrow (a+2) \rightarrow (m-1) \rightarrow 0 \rightarrow (a-1)$

Scanned with CamScanner

of probe sequences possible = m

Let's see using diff values of a

a=0	0, 1, 2	———	m-1
a=1	1, 2, 3	———	m-1, 0
a=2	2, 3, 4	———	m-1, 0, 1
	⋮		
	⋮		
a=m-1	m-1	0, 1, 2, ———	m-2

} m probe sequences possible

Secondary clustering → If two probes are found to have same initial value then they will follow same probe sequence.

Primary clustering - Build of sequence/continuous run of items, eg



Ideal Prob = $\frac{1}{n}$

Prob = If's own prob + if element arrives at above 3 also

= $\frac{4}{m}$

This is called primary clustering i.e if we have continuous runs of items the prob of immediate empty position increases.

Primary Clustering deteriorates search time

Crack 2007

Keys: 12, 18, 13, 2, 3, 23, 5 & 15

Hash Table size = 10

$h(K) = K \bmod 10$ & linear probing

What is resultant Hash table.

Ans

$12 \bmod 10 = 2$

$18 \bmod 10 = 8$

$13 \bmod 10 = 13$

Now 2 will follow linear probing

$h'(2,0) = (h(2)+0) \bmod 10$

$= (2+0) \bmod 10 = 2$

$h'(2,1) = (2+1) \bmod 10 = 3$

$h'(2,2) = 4$

My keep going

0	
1	
2	12
3	13
4	2
5	3
6	23
7	5
8	18
9	15

Crack 2008: HTS = 11 (0–10) $h(K) = K \bmod 11$

Keys: 43, 36, 92, 87, 11, 4, 71, 13, 14

linear Probing

What is index into which last word is inserted

Ans

0	87
1	11
2	13
3	36
4	92
5	4
6	71
7	14
8	
9	
10	43

→ Ans

Scanned with CamScanner

Gate 2007 Page 205

Q-1 Consider a hash function that distributes keys uniformly. HTS = 20. After hashing of how many keys, will the probability that any new key hashed collides with an existing one exceed 0.5

a) 5 b) 6 c) 7 d) 10

Ans 

Collision prob = $\frac{i}{m}$

$\frac{i}{m} > 0.5$ $\frac{i}{20} > 0.5$ $i > 10$

Ans 11

Q2 h(k) = k mod 10 linear probing

A) a) 46, 34, 42, 23, 52, 33

b) 34, 42, 23, 52, 33, 46

c) 46, 34, 42, 23, 52, 33

d) 42, 46, 33, 23, 34, 52

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

B) How many insertion sequences are possible.

Ans - a) ✗ c) ✓

b) ✗ d) ✗

B) 42 23 34 can come in any manner but they should come first

52 should come after 31 = 6

33 definitely after 46, 42, 23, 34, 52

always last

42 23 34 52 33

46 can come at any 5 of 31 = 30 ways

Quadratic Probing

Linear - $h'(K,i) = (h(K) + i) \bmod m$

Quad - $h'(K,i) = (h(K) + c_1 i + c_2 i^2) \bmod m$ such that all are covered

Advantage :- No primary clustering.

Disadvantage :-
1) But secondary clustering will be there
2) Very tricky to choose c_1 & c_2
3) Here also Prob sequence = m (Parallel)

Double hashing

$h'(K) = (h_1(K) + i\, h_2(K)) \bmod m$

Adv 1) No secondary clustering
2) No primary "

Linear Prob seq = m
Quad " " = m
Double hashing " " = m^2

$\therefore h_1(K_1) \neq h_2(K_2)$

h_1 — 0 — m-1
h_2 — 0 — m-1 so m^2

Scanned with CamScanner

Date / Page 207

To further increase effectiveness we can extend double hashing to triple hashing and so on.

P.S → Here also m probes should land to all m locations.

To make sure this happen

1) $(h_2(k), m)$ → should be relatively prime
   Let's say both are in power of 2^k
   So m - even, $h_2(k)$ - odd so relatively prime
   or m → prime
   $h_2(k) < m$

Scanned with CamScanner